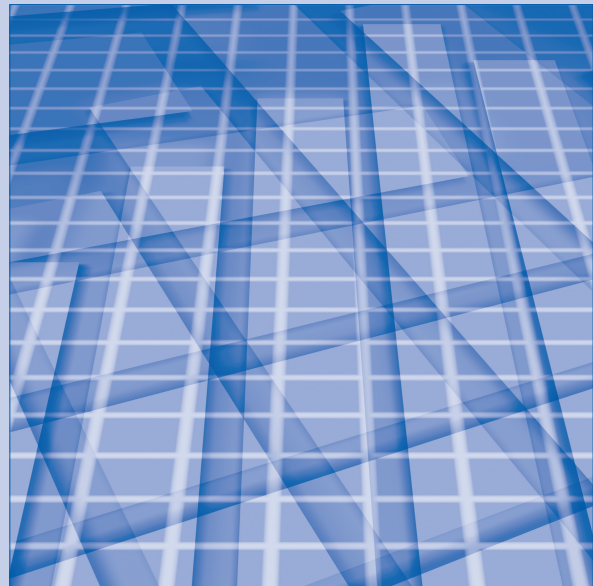




M359 Block 5

UNDERGRADUATE COMPUTING

Relational databases: theory and practice



Further database
features

Block 5

This publication forms part of an Open University course M359 *Relational databases: theory and practice*. Details of this and other Open University courses can be obtained from the Student Registration and Enquiry Service, The Open University, PO Box 197, Milton Keynes MK7 6BJ, United Kingdom: tel. +44 (0)870 333 4340, email general-enquiries@open.ac.uk

Alternatively, you may visit the Open University website at <http://www.open.ac.uk> where you can learn more about the wide range of courses and packs offered at all levels by The Open University.

To purchase a selection of Open University course materials visit <http://www.ouw.co.uk>, or contact Open University Worldwide, Michael Young Building, Walton Hall, Milton Keynes MK7 6AA, United Kingdom for a brochure: tel. +44 (0)1908 858793; fax +44 (0)1908 858787; email ouw-customer-services@open.ac.uk

Sybase, iAnywhere and SQL Anywhere are trademarks of Sybase, Inc.; Java is a trademark of Sun Microsystems, Inc. Other product and company names may appear in the M359 course material. Rather than use a trademark symbol with every occurrence of a trademarked name, we use the names only in an editorial fashion and to the benefit of the trademark owner, with no intention of infringement of the trademark.

The Open University
Walton Hall, Milton Keynes
MK7 6AA

First published 2007.

Copyright © 2007 The Open University

All rights reserved. No part of this publication may be reproduced, stored in a retrieval system, transmitted or utilised in any form or by any means, electronic, mechanical, photocopying, recording or otherwise, without written permission from the publisher or a licence from the Copyright Licensing Agency Ltd. Details of such licences (for reprographic reproduction) may be obtained from the Copyright Licensing Agency Ltd, Saffron House, 6–10 Kirby Street, London EC1N 8TS; website <http://www.cla.co.uk>.

Open University course materials may also be made available in electronic formats for use by students of the University. All rights, including copyright and related rights and database rights, in electronic course materials and their contents are owned by or licensed to The Open University, or otherwise used by The Open University as permitted by applicable law.

In using electronic course materials and their contents you agree that your use will be solely for the purposes of following an Open University course of study or otherwise as licensed by The Open University or its assigns.

Except as permitted above you undertake not to copy, store in any medium (including electronic storage or use in a website), distribute, transmit or retransmit, broadcast, modify or show in public such electronic materials in whole or in part without the prior written consent of The Open University or in accordance with the Copyright, Designs and Patents Act 1988.

Edited and designed by The Open University.

Typeset by The Open University.

Printed and bound in the United Kingdom by Martins the Printers, Berwick-upon-Tweed.

ISBN 978 0 7492 1577 4

CONTENTS

Contents	3
Introduction to <i>Block 5</i>	6
TOPIC 1 XML AND DATABASES	7
1 Introduction	8
1.1 Documents and markup languages	8
1.2 Data and XML	11
2 XML concepts	13
2.1 Structure of XML documents	13
2.2 Processing XML documents	21
2.3 XML activities	26
2.4 Summary	28
3 Using XML with databases	29
3.1 Comparing XML and relational data	29
3.2 Transforming relational data into XML	30
3.3 Storing XML data in a relational database	35
3.4 Querying XML values in a relational database	40
3.5 Activities	42
3.6 Summary	45
Topic summary	46
Solutions to Topic 1 Exercises	47
TOPIC 2 APPLICATION DEVELOPMENT	49
1 Introduction	50
1.1 The need for application processes	50
1.2 Varieties of application process	51
1.3 The rest of this topic	52
2 Embedded SQL	53
2.1 Programming languages using embedded SQL	53
2.2 Processing embedded SQL	54
2.3 Embedded SQL statements	56
2.4 ODBC (Open Database Connectivity)	59
2.5 Summary	62

3	Java and databases	64
3.1	JDBC	64
3.2	SQLJ	75
3.3	Other roles for Java	77
3.4	Summary	82
4	Alternative approaches	83
	Topic summary	87
	Solutions to Topic 2 Exercises	88
	Index	92

M359 COURSE TEAM

This course was produced by the following team (affiliated to The Open University, unless otherwise stated):

Course team

Kevin Waugh Course Team Chair and Author
Ian Cooke Author
Mike Newton Author
Judith Segal Author
Steven Self Author
Alistair Willis Author
Kay Bromley Academic Editor
Ralph Greenwell Course Manager and Accessibility Consultant

External assessor

Barry Lowden University of Essex

Critical readers

Sue Barrass
Peter Blachford
Terry Burbidge
Pauline Butcher
Pauline Curtis
Hugh Darwen
Ivan Dunn
Gillian Mills
Ron Rogerson

LTS Media team

Andrew Seddon Media Project Manager
Steve Rycroft Editor
Andrew Whitehead Designer and Graphic Artist
Phillip Howe Composer
Kamy Yazdanjoo Software Developer
Sue Stavert Technical Testing Team

Thanks are due to the Desktop Publishing Unit of the Faculty of Mathematics and Computing.

Introduction to *Block 5*

In this block, we examine two topics that extend the view of database technology as presented in the rest of the course. The first topic is concerned with XML and databases – what is special about XML and how it impacts on databases. The second topic is about application development for databases – what is involved in writing application programs that require access to a database.

The two topics are independent of each other; so the numbering of sections and so on are kept separate within each part of this block. They do not have to be studied in the order as presented in this document – you should read the *Study Guide* for details and advice on how these topics will be assessed and the appropriate order that you should follow.

Topic 1 XML and databases

1

Introduction

This topic is about the use of XML with databases, and specifically relational databases. XML is a relatively new technology, being first specified by the World Wide Web Consortium (W3C) in 1998, but it has developed rapidly both in the technology itself and in its application to many areas of computing. Whatever the application, XML may be obtained from a database or may need to be stored in one, which is why we are including the topic in this course.

We do not assume that you know anything about XML. So this section provides an introduction to the basic ideas of XML in order to establish a basis for Section 2, which reviews the essential concepts of XML. Then, in Section 3, we examine the various ways in which database management systems can support XML. We must emphasise that we do not attempt to teach you about the general use of XML, but just focus on its relevance to databases.

1.1 Documents and markup languages

The term **XML** stands for Extensible Markup Language, which is sometimes written as eXtensible Markup Language to make the derivation of the acronym explicit. In order to understand the purpose of XML, we look first at the general concept of a **markup language**, which is a notation used to label the different parts of a document with various kinds of characteristic that may be used in processing the document.

For an example, let us consider Figure 1.1 as a document which, in this case, is just a sequence of characters that you might keep in a text file. It is hoped that you can see how this minimal document is organised so that it can be extended with arbitrary amounts of text in each paragraph, many paragraphs in each section and many sections in the document.

```
Simple Document
First Section
This is the first of three paragraphs of the first section.
This is the second paragraph of the first section.
This is the third paragraph of the first section.
Second Section
This is the only paragraph of the second section.
```

Figure 1.1 The Simple Document example

The document in Figure 1.1 has no markup, it is just narrative text – you can think of it as a collection of words put together for some reason, such as telling a story. In considering what you might want to do with such a document, your initial thought might be to make it look more readable or appealing when printed or displayed on a screen, as shown in Figure 1.2.

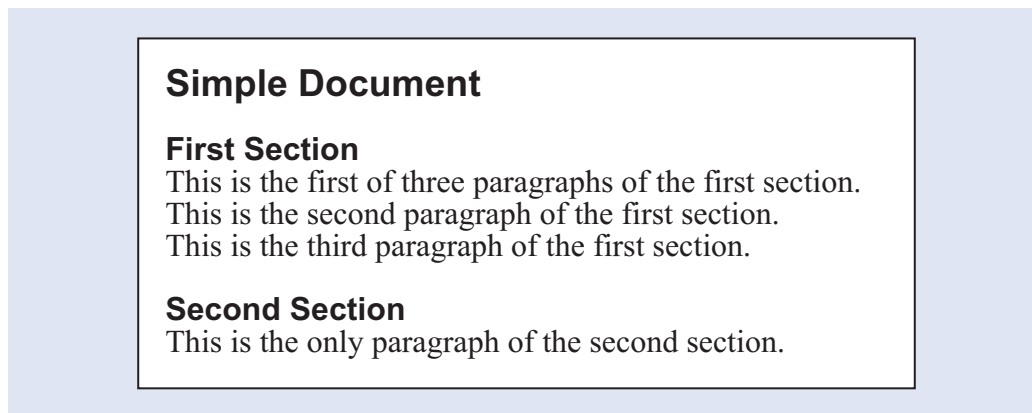


Figure 1.2 Possible formatting of the Simple Document example

You can use a word processor to do this formatting. Alternatively, HTML, the Hypertext Markup Language developed by W3C, can be used in a similar way to determine how web documents are displayed in a browser. HTML uses a set of **markup tags**, labels such as `<h1>` or `<h2>` for headings or `<p>` for paragraphs, which are embedded in the text to distinguish the different parts of the document and hence determine how each part should appear on a screen.

HTML is an application of the International Standard SGML, the Standard Generalized Markup Language, which is just that – a very generalised specification of how to mark up a document. HTML conforms to the SGML specification for markup and defines the tags that can be used for web pages, but it has one significant limitation – the tags that you can use are fixed by the W3C specification. Thus there are several levels of heading and, while each time a heading is used it can be given a different appearance, HTML cannot be used to specify headings with different meanings. For example, you can see from the text of this block that we distinguish the different headings, such as section headings, exercise headings, and so on. This is not possible with HTML.

In attempting to capture and understand the meaning of a document, it is usually more important to represent its logical structure than it is to specify what it looks like. SGML does provide this capability, but it is complex and is used mainly by specialist publishing experts. XML is a subset of SGML that was defined by W3C for use on the Web, providing many of the same capabilities as SGML, but it is much easier to use. This has resulted in the general adoption of XML for various purposes, many of which are beyond the original scope of markup of web documents, as we shall consider shortly.

To see how XML can be used to mark up the structure of a document, Figure 1.3 shows a possible markup for the Simple Document given in Figure 1.1 (though it is incomplete, as we consider in Section 2).

```
<my-title>Simple Document</my-title>
<my-section my-number='1'>
  <my-heading>First Section</my-heading>
  <my-para>
    This is the first of three paragraphs of the first section.
  </my-para>
  <my-para>
    This is the second paragraph of the first section.
  </my-para>
  <my-para>
    This is the third paragraph of the first section.
  </my-para>
```

```
</my-section>
<my-section my-number='2'>
  <my-heading>Second Section</my-heading>
  <my-para>
    This is the only paragraph of the second section.
  </my-para>
</my-section>
```

Figure 1.3 XML markup for the Simple Document (incomplete)

In addition to the text in Figure 1.1, Figure 1.3 includes embedded markup that encloses each part of the text by a pair of tags: `< ... >` and `</ ... >`. These tags provide the structure of the document as having a title followed by a number of sections, and each section contains a heading and some paragraphs. We will examine this structure in more detail in Section 2, and complete its specification as an XML document, but for now it is enough to appreciate that the structure is determined by the tags, such as `<my-section>` and `<my-para>`. However, from the names we have used you should appreciate that these tags are determined not by XML but by us in choosing how we want to structure the document.

Thus, while XML is called a 'language', it does not provide you with a vocabulary of terms as do other languages – the first steps in learning most languages is to find out about the basic words, such as 'Bonjour' or 'Dzien dobry'. What XML does provide for you is a grammar – a set of rules that tell you how to write correct XML – together with the means to define your own vocabulary, that is, the terms you want to use for the tags in structuring your documents. Consequently, XML is sometimes referred to as a 'metalanguage', that is, a way of defining the terminology for another new language, which is why XML is called an extensible markup language.

While our example has introduced just a few terms for describing the Simple Document, it is hoped you can appreciate that in practice a real document, such as the text you are reading now, has a more complex structure that requires a very much larger set of terms to enable you to describe every detail. The author of such a document would not be expected to produce their own set of terms, and there are various sets of document definitions that can be used for this purpose. Some examples are DocBook, which is intended for technical writing, and Text Encoding Initiative (TEI), which is intended for more literary works. For more specialist writing there are, for example, MathML (Mathematics Markup Language) and CML (Chemical Markup Language). Each of these is an XML application, defining a set of terms – a vocabulary – to be used as the markup for structuring a document. Whatever the application, the result is referred to as an XML document and is contained in a file whose name usually has the extension `.xml`. Such a file only contains textual characters and so can be read by any text editor.

Producing an XML document does not necessarily require a person to explicitly write all the detailed markup as shown in the example in Figure 1.3. Instead of using a text editor, there are specialised XML editors that, given the definitions for an XML application such as DocBook, will enable you to enter your text and structure it without you having to know the detailed XML syntax. Furthermore, it is important to remember that the main purpose of markup is to support the processing of a document in various ways. One example of this is SVG (Scalable Vector Graphics), which defines an XML vocabulary for specifying graphics. This XML would be produced by a graphics program to represent shapes drawn by a user, which could then be displayed by some other program.

Notice that SVG demonstrates an important benefit of using a defined vocabulary – that is, such a document is 'portable', which means it can be moved around between different processes that have been written to accept the vocabulary.

More generally, we have noted that an XML document may be presented in different formats, such as printed pages and web pages, but there are many other processes that may be required. For example, although some processes are necessary and adequate for generating contents pages or indexes, a more specialised process may be needed to enable text to be moved from one location to another in the final layout of a document.

1.2 Data and XML

Our introduction so far has been in terms of documents that are mainly concerned with the organisation of narrative text. This is a reflection of the early development of XML from SGML. However, its major growth in recent years has been associated with different kinds of ‘documents’ – ones with greater variety in the type of data they contain and the way they are organised. In particular, this kind of application of XML has been associated with what is known as ebusiness – that is, the way organisations conduct their business via the exchange of electronic messages, usually over the internet. Some examples of XML definitions that provide a vocabulary for this kind of application include ebXML (providing a number of different capabilities for electronic business) and XBRL (Extensible Business Reporting Language), as well as more specialised ones, each targeted at a single industry such as automotive or telecommunications.

In addition to these defined application vocabularies, in the same way that we created our own XML vocabulary for the Simple Document shown in Figure 1.3, it is possible to create a vocabulary for any other application. Let us consider an XML application derived from the University example that is used in the course. Suppose there is a requirement for the University to provide a record of a student’s courses to an external organisation, such as an employer who is paying for the courses on which the student is enrolled. This record is often referred to as a transcript, which can include various kinds of information, such as the results for those courses. However, we will focus on the essential XML and not complicate it with too much detail.

Figure 1.4 shows an example of a transcript for student s05, with XML tags that we have introduced for this purpose. At this stage, we do not intend providing an explanation of the XML structure, which we shall consider in Section 2, but we will use this example to demonstrate some issues about the use of XML.

```
<studentTranscript>
  <studentId>s05</studentId>
  <studentName>Ellis</studentName>
  <enrolments>
    <course>
      <courseCode>c2</courseCode>
      <title>Syntax</title>
      <credit>30</credit>
    </course>
    <course>
      <courseCode>c7</courseCode>
      <title>Pragmatics</title>
      <credit>30</credit>
    </course>
  </enrolments>
</studentTranscript>
```

Note that we also refer to this kind of representation of data as an XML document, even though it does not contain narrative text that is usually associated with documents.

Figure 1.4 Example student transcript as an XML document

First, in Figure 1.4 we have presented the document in a two-dimensional layout that enables us to show the structure of the document by lines and indentation, whereas it should be considered as a sequence of characters whose structure is represented entirely by its markup – spaces and new lines are not significant.

Secondly, it can be seen that the data content of this document – that is, what remains after removing the markup – corresponds to the data relating to student s05 in the University database, though not just data from the **student** table but also data from the **enrolment** and **course** tables. So it is necessary to consider how such an XML document is produced.

Thirdly, you need to think about what happens when this document arrives at the organisation for which it was produced. Since it is just a sequence of characters, the document can be read. However, while it is XML, that does not mean that the terms, such as **enrolments**, used for the markup will necessarily be understood by the people in that organisation. For these kinds of messaging applications, it is not enough just to say you are using XML, it is also necessary to ensure that all parties involved know and understand the terms that are being used to markup the XML documents.

Finally, the receiving organisation is likely to receive transcripts for many students and, therefore, it needs some way of processing the XML document to extract and store the relevant data in its own database.

Thus the University that produces the XML document and the organisation that receives it both need a program that can process XML according to their requirements. In the XML context, such a program is referred to as an **XML application**.

We are not going to consider the details of writing XML applications here, but in Section 3 we shall examine the facilities that can be provided by a DBMS to support XML applications that involve a database.

Before we can consider what a DBMS can do with XML, we need to examine the general characteristics of an XML document, how to define its properties and what kinds of processing may be required. We shall do this in the next section.

2

XML concepts

There are a number of different constituents of an XML document that represent both the structure of its content and how it can be processed. As we have already mentioned, the main purpose of markup is to support the processing of a document, but we will consider that in more detail after we have examined the structure.

2.1 Structure of XML documents

At the most basic level, an XML document is a sequence of characters, which is partitioned into groups that are treated as either markup or **character data**. We first examine how markup is distinguished from character data, and how it is used to separate the character data into components that represent the meaningful structure of the document.

Elements

The main structure of a document is determined by its **elements**, where an element is bounded by a **start-tag**, formed by `<element-name...>` (at this point, ignoring what else can appear between the angle brackets), and an **end-tag**, formed by `</element-name>`.

A tag's `element-name` is a label that begins with a letter and continues with letters, digits or symbols such as a hyphen, underscore or full stop (a colon is also allowed but it has a special meaning, therefore you should not use it).

Whatever appears between a start-tag and its matching end-tag is the content of the element, which may be either character data or other elements. It is also possible to have an **empty element** that has no content, in which case it may be written using a single tag `<element-name.../>`, where the ending `/>` signifies that there is no matching end-tag.

In Figure 1.4, an example of an element is `<title>Syntax</title>`, where the element name is `title` and the character data content of the element is `Syntax`. A more complex example of an element is given by the whole figure, beginning with the start-tag `<studentTranscript>` and finishing with the matching end-tag `</studentTranscript>`. In this case, the element name is `studentTranscript` and its content between the start-tag and end-tag is considered as the sequence of elements named `studentId`, `studentName` and `enrolments`, each of which has its own content.

The idea that an element can contain other elements – that is, they are nested – is the main way in which an XML document is structured. This can be expressed in terms of a data structure known as a **tree**, which is constructed as a connected set of **nodes** with the following rules. Each **parent node** may be connected to one or more **child nodes**. Contrary to the use of these terms for humans, in a tree a child node can have only one parent. Furthermore, a tree must have one node that does not have a parent; this is known as the **root node**. A node that does not have any child node is a **leaf node**.

Note that an element name is case sensitive, which is different from the naming of SQL tables and columns.

A tree is a particular kind of **hierarchical data structure**.

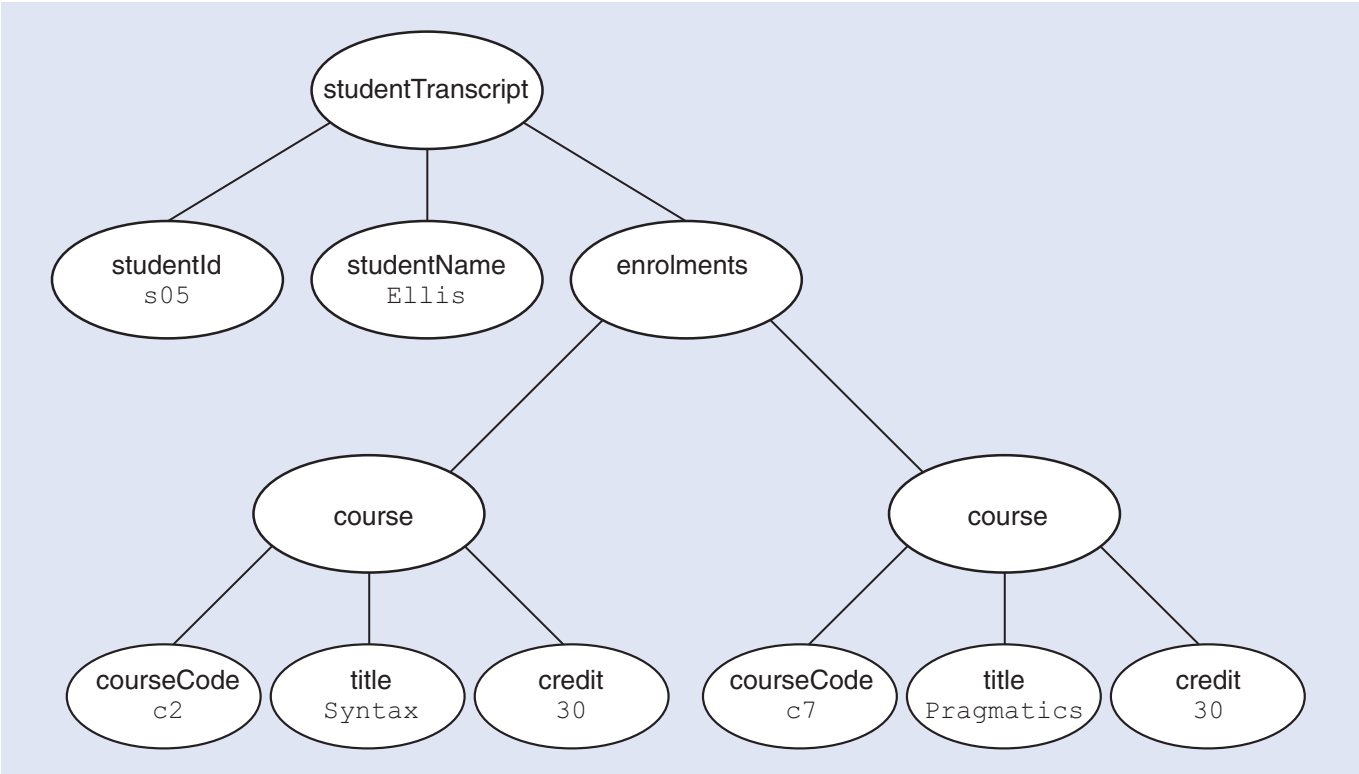


Figure 2.1 Tree structure for the student transcript example

For XML, we can consider elements as the nodes of a tree. Figure 2.1 shows the tree structure of the student transcript from Figure 1.4, where we follow the convention of putting the root at the top of the tree and a child is always below its parent.

The tree structure in Figure 2.1 shows the elements from Figure 1.4 with their associated character data contents. In some situations, an XML document is represented by a tree structure in which there are separate nodes for the character data content of each element, but then you need to remember that it is character data and not an element.



EXERCISE 2.1

List the leaf nodes in Figure 2.1.

While the tree structure of an XML document may appear obvious when shown in a diagram like Figure 2.1, it is not so clear when you consider the text from which it originates. In Figure 1.4 we have tried to help you envisage the structure by indentation of the different levels in the tree, but that is not relevant to the way the structure of the document is defined by the tags that determine its elements. As a character string it would just be as follows.

```
<studentTranscript><studentId>s05</studentId><studentName>Ellis
</studentName><enrolments><course><courseCode>c2</courseCode>
<title>Syntax</title><credit>30</credit></course><course>
<courseCode>c7</courseCode><title>Pragmatics</title><credit>30
</credit></course></enrolments></studentTranscript>
```

When seen in this form, the tree structure is not at all clear. The following rules ensure that a document has elements that are properly structured as a tree. A document obeying these rules is said to be **well-formed XML**.

- ▶ Every start-tag must have a matching end-tag, and vice versa, that determines an element and its contents, which is whatever appears between the tags.
- ▶ A document has one root element.
- ▶ Every other element must be contained within some parent element, requiring that each pair of matching tags is nested within another pair (i.e. if a start-tag is in the content of an element, then its end-tag is in the content of the same element).

EXERCISE 2.2

In Figure 1.3, explain why the markup given for the Simple Document does not satisfy these rules for a tree of elements, and suggest what can be done so that it satisfies the rules.



EXERCISE 2.3

Give the tree structure (in the style of Figure 2.1) for the Simple Document in Figure 1.3 with the addition of the root element described in the solution to Exercise 2.2. You do not need to include the character data content.

There is one final feature that we need to consider in the precise correspondence between an XML document and its tree representation – that an XML document is just a sequence of characters. This sequential nature is obviously intrinsic to narrative texts, where it is essential to be able to read a story in the right order, though it is not always necessary for data-oriented applications. So let us consider how this feature of an XML document is represented in a tree structure and, conversely, how the sequential character representation of an XML document can be recreated from a tree structure – this is called **serialization** of the tree.

Maintaining the order of elements in a tree structure requires that each collection of child nodes for any one parent node is kept in order. We have expressed this diagrammatically by simply showing the order of the child nodes as left to right. To show that this represents the order of elements for the whole document, you need to consider how a tree is serialized in terms of moving from one node to another, which is called a **traversal** of the tree.

At any node you can have several choices as to which node to go to next – to any of the children of that node or to its parent. The specific traversal for an XML document is based on moving from a node to each of its children in turn (remember they are ordered) and, when there is no other child, you return to the parent. This process starts at the root element and is applied from one node to another so that, for each child, move to its children, and so on until you return to the root node.

EXERCISE 2.4

List the nodes visited by the tree traversal applied to Figure 2.1, giving its serialization.



You should check this solution against the original XML version given in Figure 1.4 to see that the traversal has reproduced the same sequence of elements. This tree traversal also works if the character data content of each element is treated as a separate node.

Before leaving this examination of elements, we need to remind you of one comment about the XML document given in Figure 1.4 – that we specified the tags for the document – so we had to make choices in doing so. These choices are not just about the names we might use for the elements but also about what elements, and hence

what structure, is appropriate. We could have taken a view that the element `enrolments` was not necessary as a way of grouping the courses on which a student is enrolled, which would have led to a document as shown in Figure 2.2.

```
<studentTranscript>
  <studentId>s05</studentId>
  <studentName>Ellis</studentName>
  <course>
    <courseCode>c2</courseCode>
    <title>Syntax</title>
    <credit>30</credit>
  </course>
  <course>
    <courseCode>c7</courseCode>
    <title>Pragmatics</title>
    <credit>30</credit>
  </course>
</studentTranscript>
```

Figure 2.2 Alternative XML for the student transcript

You should check that the XML in Figure 2.2 has the same character data as Figure 1.4, and maintains the same organisation of that data content. With such alternative representations from which to choose, you need to have some criteria to decide which would be most appropriate in a given situation. It is not the concern of this course to start addressing these design issues; in general, it depends on both representing the meaning of the data content and supporting the required processing. For example, while removing `enrolments` from Figure 1.4 did not result in loss of meaning, if we removed the element `course` in a similar way it would leave the same data content but represented in a way that does not show the grouping of `courseCode`, `title` and `credit` as the data that is needed for a course.

Let us now consider another kind of constituent of XML documents.

Attributes (of elements)

While the main structure of an XML document is determined by its elements and their content, an element can also have properties defined by **attributes**. The (incomplete) XML document in Figure 1.3 includes an example of an attribute for the element `my-section`, where an attribute named `my-number` is assigned the value 1 in the start-tag `<my-section my-number='1'>`. This attribute was something we added to the text from Figure 1.1 in creating the markup shown in Figure 1.3 and, apart from being an example to which we can refer, you can envisage its use as a numeric label for the sections of the document.

In general, an element can have several attributes, each of which has a different name. These attributes are specified within the start-tag for the element by the combination of a name and a value. An attribute name is formed in the same way as we described for an element name. An attribute value must be enclosed by either a pair of single quotation marks, `'...'`, or a pair of double quotation marks, `"..."`, but not both of them.

You can use an attribute for any kind of value, but remember that it must be written as a character string; so the integer 1 is given as `'1'`. If you have a collection of data to be represented as an XML document, you will have to decide which values are to be represented as attributes of an element and which values are to be represented as character data within an element. Just to demonstrate this choice, we can again use the data for the student transcript. Figure 2.3 shows how attributes can be used in this case.


```
<studentTranscript>
  <studentId>s05</studentId>
  <studentName>Ellis</studentName>
  <enrolments>
    <course courseCode="c2" title="Syntax" credit="30"/>
    <course courseCode="c7" title="Pragmatics" credit="30"/>
  </enrolments>
</studentTranscript>
```

Figure 2.3 Using attributes for courses in the student transcript

In the XML document in Figure 2.3, there are two elements named `course`, each with the attributes named `courseCode`, `title` and `credit` with their associated values. These `course` elements are examples of an empty element, since they do not have any content, just attributes. Note that the ordering of attributes is not significant (contrary to what we said about elements) so we could have written the attributes of each of the two `course` elements in different orders. As we shall see, processing this document selects the attributes by name, which is why attribute order within each element does not matter.

So, in choosing an XML representation of your data, as well as having to decide what elements are required, you also have to decide what attributes might be appropriate. There are many opinions on this matter and such choices are not an issue that we can consider in this course, but there are some observations that may help.

First, each attribute of an element may have only one value. For example, if we simplified the student transcript so that only the course code was required for each course, we could not have `courseCode` as an attribute of the element `enrolments` because it would need two values in this example. However, it would be possible to have an attribute of `enrolments` named, say, `courseCodes` with a value of `"c2, c7"`, but that is not so clear as having the courses as separate elements.

Secondly, while an attribute value can be any length of character string, when viewing the markup for a document it is beneficial for an attribute to be fairly short because it makes it easier to see that the attribute value is embedded within the start-tag of the element to which it belongs.

Thirdly, it must always be remembered that an XML document will be processed in some way, and so the choice of attributes is influenced by what you might want to do with the data.

Arising from this final point, there is one kind of data that should be treated differently from normal application data; it is known as metadata – that is, data describing other data. In the case of an XML document, attributes can be useful for keeping metadata describing the content of an element separate from the content itself. For example, the root element for the Simple Document, as described in the solution to Exercise 2.2, could have attributes for the author of the document and the category under which it should be classified, so its start-tag may be expressed as something like

```
<my-simple author="Jo Bloggs" category="final-report">
```

The example in Figure 2.3 demonstrates that the data content of an XML document – that is, the data that is not markup – is not only the character data contained in the elements but may also include attributes of the elements. However, most of our examples do not have attributes, so character data will normally be the data content of an XML document, and we use the terms synonymously.



EXERCISE 2.5

Give a start-tag for the `studentTranscript` element with attributes showing that the transcript originated at Walton University on 29 October 2006.

Defining documents

So far in this section, we have described how the markup in an XML document must obey certain rules that determine how the elements and their attributes should be written, but otherwise a document has not been constrained as to the types of element that are allowed – that is, we have been able to choose any naming and organisation of the elements.

However, there are many reasons why a definition of a document might be required to specify exactly what types of element are allowed. In Section 1 we mentioned some different kinds of document definition, varying from TEI for narrative texts to ebXML for ebusiness. But such a definition is also necessary in more limited applications where a document is to be shared, for example, as we have described for the student transcript, because that is the way to ensure it is understandable to everyone involved as well as constraining how a document is created and processed. An XML document that conforms to a document definition is said to be **valid XML**.

There are several ways in which an XML document may be defined. The original means of defining a document is given as part of the XML specification and is known as a **document type definition** (DTD). Figure 2.4 shows an example of a DTD for the Simple Document given in Figure 1.3, with the addition of the root element as described in the solution to Exercise 2.2.

```
<!ELEMENT my-simple (my-title, my-section*) >
<!ELEMENT my-title (#PCDATA) >
<!ELEMENT my-section (my-heading, my-para*) >
<!ATTLIST my-section my-number CDATA #REQUIRED>
<!ELEMENT my-heading (#PCDATA) >
<!ELEMENT my-para (#PCDATA) >
```

Figure 2.4 A DTD for the Simple Document example

While we are not going to consider the rules of constructing a DTD in detail, we can use the example shown in Figure 2.4 to demonstrate some basic features. The lines of this DTD example specify that there are five types of element and one attribute, which can be interpreted as follows:

- ▶ the element `my-simple` contains one `my-title` element and possibly many (indicated by `*`) `my-section` elements, in that order;
- ▶ the element `my-title` contains parsed character data (indicated by `#PCDATA`);
- ▶ the element `my-section` contains one `my-heading` element and possibly many `my-para` elements, in that order;
- ▶ the element `my-section` has an attribute `my-number`, which is required to have a value that is character data (indicated by `CDATA`, and it is not parsed);
- ▶ the element `my-heading` contains parsed character data;
- ▶ the element `my-para` contains parsed character data.

There are many other ways in which a DTD can define the structure of an XML document, allowing elements to have a very varied content, such as containing both character data and elements (known as mixed content) or containing elements that can be optional or without ordering. There are more limited variations for attributes, since they have just a single character data value; for example, they can be optional and can have a default value.

We elaborate on parsing when considering XML processing in Subsection 2.2.

EXERCISE 2.6

Give the DTD specification for the root element in Figure 2.2.



Apart from demonstrating how a DTD determines the structure of the elements within an XML document, there are other properties of DTDs that should be noted from Figure 2.4. One property that can be observed is that a DTD is not itself an XML document. While Figure 2.4 has many `<` and `>`, it does not conform to the way tags are written for well-formed XML.

A second property is that the data defined by a DTD for both element content and attribute values is always character data. There is no way to define, for example, that a course credit has to be an integer or that the attribute `date` in the solution to Exercise 2.5 must have a legitimate date value.

A third property of a DTD, which is not immediately obvious from Figure 2.4, is that its ordering does not matter – you can mix up the element and attribute definitions and it does not affect their meaning, since it still determines the same structure.

Finally, one property of a DTD that we have not demonstrated in the example is the ability to define an **entity**, which is a named value that can be included within character data by giving its name embedded between `&` and `;`. When the document is parsed (this is discussed further in Subsection 2.2) the name is replaced by its value.

For example, we could define an entity named `dbms` in a DTD as follows:

```
<!ENTITY dbms "database management system">
```

To use this entity, you would write `&dbms;` and a parser would replace it by the character string `database management system`.

You can define your own entities for convenience, but what is much more important is that XML has five predefined entities, which are as follows:

- ▶ `<`; gives the value `<`
- ▶ `>`; gives the value `>`
- ▶ `&`; gives the value `&`
- ▶ `'`; gives the value `'`
- ▶ `"`; gives the value `"`

When the character data needs to contain any of these five symbols (`<`, `>`, `&`, `'` and `"`), it is essential that you use the corresponding entity to avoid confusion with the use of these symbols in specifying tags, character strings and entities themselves.

While DTDs are part of the XML specifications, and continue to be used for many purposes, there are other ways to define XML documents, each aiming to overcome some of the limitations of DTDs for certain kinds of application. We briefly note the alternatives to DTDs below.

The main alternative developed by W3C is called **XML Schema**, which provides the means to define the structure of an XML document in a way that is different from a DTD as well as specifying a range of data types that can be used to define the values that are allowed within an XML document. An XML schema is itself an XML document, with its own schema to determine how it should be written. While this kind of schema meets the requirements of data-oriented applications, it is considered too complex for some uses of XML.

By contrast, Relax NG is a schema language that is an evolution and generalisation of a DTD. It focuses on defining the structure of an XML document, though it can also

Note that in the text we will refer to the value of `'` as a single quotation mark and the value of `"` as a double quotation mark.

define values by using the data types specified for XML Schema. A Relax NG schema can be written either as an XML document or with a more compact syntax.

Schematron is another kind of schema language, which is based on writing assertions about the tree structure of an XML document.

There is one point about all these document definitions that must be emphasised. While a database must have a schema to define its tables before you can enter data, an XML document does not need a DTD or schema – as long as its markup follows the rules, then it is legitimate, well-formed XML. We will consider the uses of DTDs and schemas shortly, when describing the processing of XML.

Document prolog

So far we have concentrated on the markup used to express the structure and data content of an XML document, all contained within the root element of the document. However, there is further markup that can be included, which is all optional but must occur before the root element – so it is a kind of header – and is known collectively as the **document prolog**.

The first markup that should be included in an XML document is an **XML declaration**, which in its most basic form is written as

```
<?xml version="1.0" ?>
```

This statement declares that the rest of the document is based on XML version 1.0. The only alternative version is 1.1, and the XML recommendation is that you should not use it unless you need the extra features of that version (which we do not, since they are fairly limited anyway, such as allowing multiple languages).

There is another property of a document that can be included in an XML declaration, which is an encoding declaration that specifies how the characters in the document are represented. Usually this is specified as Unicode, and since Unicode UTF-8 (the generalisation of ASCII code) is widely used on the internet, it is adopted as the default encoding so that you do not have to include it; if you do, it would be written as

```
<?xml version="1.0" encoding="UTF-8" ?>
```

After an XML declaration, one kind of markup that can be included in a prolog is a **document type declaration**, which specifies the document type definition (i.e. DTD) that is applicable to the document. A DTD can be specified as being internal or external to the document it defines. When it is internal, the DTD is included as part of the XML document, embedded in brackets, [...], as in the following example using the DTD for the Simple Document given in Figure 2.4.

```
<!DOCTYPE my-simple[
  <!ELEMENT my-simple (my-title, my-section*)>
  <!ELEMENT my-title (#PCDATA)>
  <!ELEMENT my-section (my-heading, my-para*)>
  <!ATTLIST my-section my-number CDATA #REQUIRED>
  <!ELEMENT my-heading (#PCDATA)>
  <!ELEMENT my-para (#PCDATA)>
]>
```

When it is external, a DTD is kept in a file and the document type declaration references the file using a system identifier. Such a system identifier can take various forms, since the DTD can be anywhere on the internet, but the simplest option is to place the DTD in the same folder as the referencing XML document, in which case you need only the file name. This is shown in the following example, where the DTD is in the file `Simple_DTD.dtd`:

```
<!DOCTYPE my-simple SYSTEM "Simple_DTD.dtd">
```

For both internal and external DTDs, the name used for `DOCTYPE` in the document type declaration must be the same as the name of the root element of the document, which in this example is `my-simple`, but there is no constraint on the name of the file for an external DTD. It is possible for a document to have both an internal and an external DTD, in which case the definitions are combined.

Another kind of markup in a prolog is called a **processing instruction** (PI). The purpose of a PI is to provide a specification concerning how an XML document may be processed, and so the first part of this markup is to identify the kind of processing to which the PI relates. There is no limit to the processing that can be considered, but an example of a common requirement is to display an XML document according to a **stylesheet** that specifies the format and appearance of the document. A PI that could be included in the Simple Document to specify which stylesheet should be used for this purpose can be written as follows, where the stylesheet is in the file `Simple_XSL.xsl`:

```
<?xml-stylesheet type="text/xsl" href="Simple_XSL.xsl"?>
```

We will elaborate on the use of stylesheets in more detail when we consider the processing of XML documents in the following subsection.

The final kind of markup in a prolog, or anywhere in an XML document except within tags, is a comment. This can be used for any purpose but is normally included for documentation, such as describing the meaning of the tags. A comment can include any text, possibly covering many lines, embedded between `<!--` and `-->`, such as:

```
<!-- This is an example comment -->
```

2.2 Processing XML documents

An XML application is a kind of program that uses XML in some way or another. At this point, we are not considering the details of what an application might be doing with the XML – whether it is just being displayed for reading or being treated as data input to an ebusiness activity. The purpose of this section is to examine the XML processing that is common to many applications, and to describe the XML-related specifications for which special utility software is widely available.

Parsing XML

We start by describing how any application can use XML without having to process all the markup that an XML document contains, enabling it to focus on the character data contained within the document. That is achieved by an application invoking the support of an **XML parser**, which is software that takes as input the sequence of characters from an XML document and analyses it to separate its markup and character data. A parser can then make the character data available to an XML application in various ways, depending on the interface it provides to applications.

While parsing is common to processing all kinds of language, XML has some particular features that we need to examine so that you can appreciate what is happening. Figure 2.5 shows a basic model of what is involved in this processing.

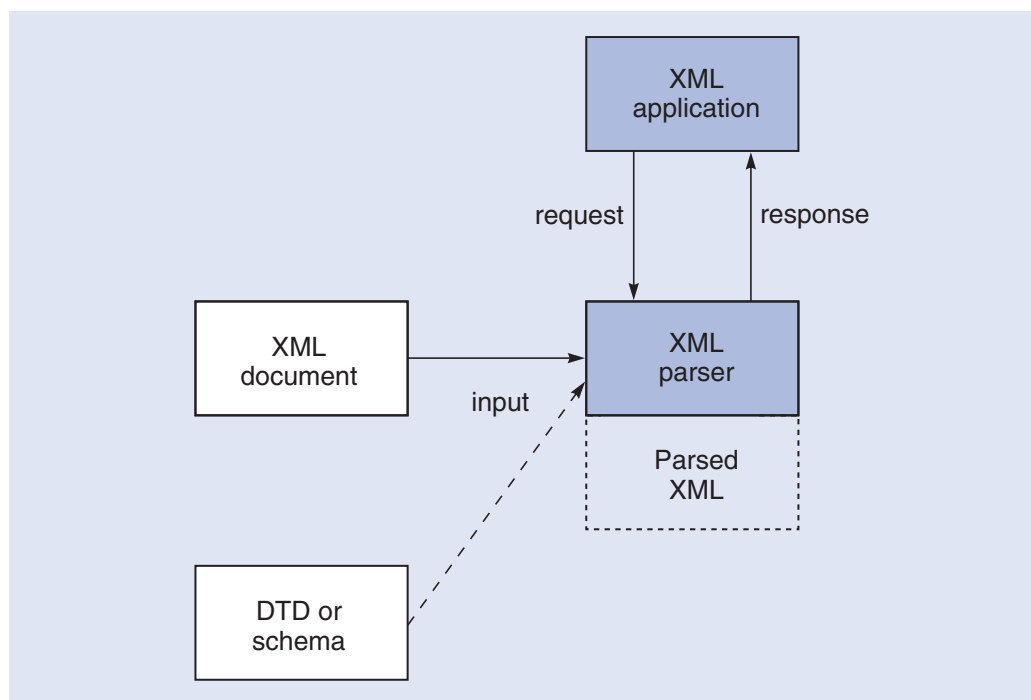


Figure 2.5 Model of XML processing

In the model given in Figure 2.5, an XML application requests that an XML parser processes a specified XML document, say in some file. The request may also specify that a DTD or schema be used. An XML parser can process that input in various ways.

In the simplest process, a parser checks that a given input document obeys the rules given in Subsection 2.1 for properly structured XML, ensuring it is well formed. If the input does not satisfy these rules it is not an XML document and no further processing is possible.

A parser can perform another kind of checking when a DTD or schema is specified. In this case, the parser assesses the given input document against the definitions in the DTD or schema to check whether the document conforms to the definitions – that is, whether it is valid.

During parsing of an XML document, its data content can be transformed in the following three ways:

- ▶ Each entity, recognised by its surrounding symbols & and ;, is replaced by its defined value.
- ▶ Each attribute that does not have a value is assigned its default value, if one has been specified in a DTD or schema.
- ▶ All sequences of whitespace characters (i.e. space, tab, carriage return and line feed) are condensed into a single space character.

When an XML application requests any of the parsed data, there are two approaches for providing the data, each with its own kind of interface to support it:

- ▶ A parser may process the input document sequentially and extract the data requested by the XML application as it does so, without retaining a copy of the parsed XML.
- ▶ A parser may process the input document and construct an internal representation of the parsed XML, which is available for further requests from the XML application.

Sequential processing is effective for single requests, but it would be inefficient for multiple requests, though for very large documents it may be the only option.

When the parsed XML is retained, it is kept as a tree structure as described in Subsection 2.1. It can be accessed by an XML application via an interface defined as the **Document Object Model** (DOM). This interface also allows the parsed tree to be updated.

Selecting XML content

While a DOM provides the means by which an XML application can request data content from an XML document via a programming interface, there are other ways to specify the elements and attributes that may be required by an application. We will briefly describe two approaches defined by W3C: XPath and XQuery.

XPath is the simpler approach and it applies to the tree structure of a parsed XML document. It is basically similar to the way files and folders can be referenced by a path (after all, files are organised in folders in a tree structure, just like an XML document). An XML tree in XPath has one significant difference from the tree of elements we described previously, in that its root is the whole document. This means that the XPath tree includes processing instructions and comments that are part of a document's prolog but, because the document is parsed, it does not include things like entities. We are not considering the details of XPath, but we will present some examples to demonstrate the capabilities it provides.

What is called an **absolute path** starts from a document root (referenced by /) and then specifies the steps to the required elements of the document. For example, consider the following XPath expression applied to the XML in the Simple Document whose tree diagram for its elements is given in the solution to Exercise 2.3:

```
/my-simple/my-title
```

This expression gives the node, that is, the element `my-title` for that document. So the result of evaluating the expression is the following XML:

```
<my-title>Simple Document</my-title>
```

If you just want the character data for that element, which in this case is `Simple Document`, you can use the function `text` as follows:

```
/my-simple/my-title/text()
```

In this example only one element is selected, but it is possible for an XPath expression to select several elements, such as:

```
/my-simple/my-section/my-heading
```

This expression gives two elements for the two section headings in the document: one for the first section and the other for the second section. If you want only the second heading, you can select it by evaluating the following expression, where 2 specifies the second section (remember all elements are ordered):

```
/my-simple/my-section[2]/my-heading
```

Another way of selecting particular elements is to use their attributes. Thus, for the Simple Document example, we can use the attribute `my-number` of the `my-section` element to select the second heading, as follows:

```
/my-simple/my-section[@my-number="2"] /my-heading
```

In this case, 2 is the value of the attribute `my-number`, which appears similar to the 2 in the preceding example (because it is used for a similar purpose) but, in general, any attribute with any value can be used in this way.

Instead of using an absolute path starting at the document root, there are processing situations that need to start at the current node – this is called the **context node**. In this

case, you can have a **context path** that starts at the context node and uses the same kinds of expressions as in an absolute path to select specific elements.

We are not elaborating any further on the many features provided by XPath for selecting parts of an XML document, since the most important issue for you is the role it plays in processing XML documents. In particular, XPath is used in XQuery, which we can now consider.

XQuery is a query language that provides comparable querying capabilities to SQL, and thus can be quite complex. However, it has one form that makes direct use of XPath, as shown in the following example:

```
doc("Simple_xml.xml")/my-simple/my-section[2]/my-heading
```

In this example, the XML document to be queried is in the file given as the argument of the XQuery function `doc`, followed by the XPath expression that selects the elements to be returned. In this case, the result of the XQuery expression is the element `<my-heading>Second Section</my-heading>`. We could have requested just the data content of the element `my-heading` by adding the function `text()`.

There is an alternative and more general way of writing XQuery that is directly comparable to SQL. This is shown in the following expression, which gives the same result as the previous example:

```
for $elemvar in doc("Simple_xml.xml")/my-simple/my-section
where $elemvar/@my-number = "2"
return $elemvar/my-heading
```

In this expression, the first line declares `$elemvar` as a variable, signified by `$`, ranging over the elements of the given document that are determined by the associated XPath expression. The second line has a condition to be applied to those elements, which uses each element given by the variable `$elemvar` as a context node for a context path. In this case, the path just leads to the attribute `my-number` with a condition that its value must be 2. The third line again uses the variable `$elemvar` as a context node with a path to specify that the element `my-heading` is to be returned.

This query is an example of a FLWOR expression, which stands for For-Let-Where-Order-Return, though in this example we have no clauses for either Let (specifying additional variables) or Order (specifying an order for elements in the result). Note that the XQuery syntax requires all keywords in lower case.

XQuery is not the only use of XPath. It is also used for XPointer – a way of one XML document referencing another – and in transforming one XML document into another, which is considered next. XPath and XQuery are also needed when we consider the use of XML in databases.

Transforming XML documents

For a given XML document, there are two distinct ways in which it can be transformed. First, an XML document can be transformed into another XML document, and the W3C specification of how to do this is known as **XSLT** (XML Stylesheet Language Transformations). Secondly, an XML document can be transformed into some other format suitable for output, such as a pdf file, and the W3C specification of how to do this is known as **XSL-FO** (XML Stylesheet Language Formatting Objects).

Work on both of these kinds of transformation originated together as XSL (XML Stylesheet Language), but they were separated because of the different purposes relating to each of them. However, they still have one thing in common, which may be observed from their title, and that is the use of a stylesheet. In both cases, the transformation involves a processor taking as input an XML document and an associated stylesheet to produce the transformed document as output. We are not

Not surprisingly, FLWOR is referred to as 'flower'.

going to consider XSL-FO any further, but we will give an example of XSLT to demonstrate the principles involved.

A particular reason to consider XSLT is that there is a W3C specification known as XHTML that is valid XML, and yet is sufficiently compatible with HTML that it can be displayed in web browsers. Thus XSLT can be used to transform an XML document into an XHTML document for viewing in some appropriate format.

To demonstrate what XSLT looks like, Figure 2.6 gives a stylesheet that can be used with the Simple Document to transform the document from XML to XHTML.

```
<xsl:stylesheet version="1.0"
  xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
  <xsl:template match="/">
    <html>
    <head>
    <title>Testing the Simple stylesheet</title>
    </head>
    <body>
      <xsl:apply-templates/>
    </body>
    </html>
  </xsl:template>
  <xsl:template match="my-para">
    <p><xsl:value-of select="."/></p>
  </xsl:template>
  <xsl:template match="my-title">
    <h1><xsl:value-of select="."/></h1>
  </xsl:template>
  <xsl:template match="my-section">
    <h3>
      <xsl:value-of select="@my-number"/>
      <xsl:text> </xsl:text>
      <xsl:value-of select="my-heading"/>
    </h3>
    <xsl:apply-templates select="my-para"/>
  </xsl:template>
</xsl:stylesheet>
```

Figure 2.6 An example of a stylesheet for the Simple Document

We are not going to examine the details of the stylesheet given in Figure 2.6, but we need to note some general observations.

First of all you can see it is an XML document (we could have included an XML declaration to make this explicit).

Secondly, it contains a mixture of elements: those prefixed by `xsl:`, which is called a namespace and determines a set of elements defined at the referenced location `http://www.w3.org/1999/XSL/Transform`; and those elements without the prefix, which are the XHTML elements that are to appear in the transformed document.

Thirdly, the stylesheet contains a number of templates that specify an element (named in quotes) to be matched during the processing of the input XML document and, when matched, give the XHTML elements to be included in the transformed document.

Fourthly, while it may not be apparent for this example, the XPath notation is used to reference elements and attributes of the input document, such as `"."`, which means the current context node.

Apart from these comments on the content of the stylesheet, the important point to remember is its purpose – that appropriate software can process the Simple Document with this stylesheet and produce an XHTML document that can be displayed in a browser. In practice, a stylesheet may be produced for a given DTD or schema and it can then be used with any XML document that is valid for the DTD or schema.

More generally, you need to appreciate that stylesheets can be used to transform an XML document into any other kind of XML document, which is applicable in an ebusiness environment where an XML message may be sent according to the terminology (i.e. DTDs or schemas) of one organisation and then transformed by a stylesheet to conform to the terminology of a receiving organisation.

2.3 XML activities

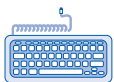
We will now look at some examples of processing XML, but we need to know which software to use. There are specialist XML editing programs that are not only capable of producing XML documents, but are also able to process them in the various ways we have just described. However, this topic is not really about XML itself, so we are not going to use this kind of software, but you will need to consider it if you want to learn more about XML.

Instead, these activities are based on the more limited XML capabilities built into the web browsers Internet Explorer (IE) and Mozilla Firefox. We have provided you with a number of files, all of which are based on the examples described in this text. You can examine them in a text editor (such as Notepad) and process them in a web browser.

The files have various extensions, such as `xml` and `dtd`, which should make their contents clear. When you are asked to open a file using a particular program (text editor or web browser), you should do one of the following:

- ▶ run the program, use the **File** menu to choose **Open** and select the file from its folder;
- ▶ open the folder containing the file, right-click on the file and choose **Open with** and select the program to be used.

At the time of writing, the browsers Netscape and Opera did not support XML processing, as described in these activities.



ACTIVITY 2.1

Open the file `Simple_XML.xml` using a text editor. You should see that it contains the same XML as shown in Figure 1.3, but with the addition of the root element, as described in the solution to Exercise 2.2, and an XML declaration and comment. Now open this file using a web browser. There are some minor differences in how each kind of web browser displays an XML file (e.g. IE includes the XML declaration but Mozilla Firefox does not); however, a common feature that you will see is the – signs to the left of the XML start-tags. If you click on one of the – signs, this will cause the associated element to be contracted to a single line, showing the element but no content, and the sign changes from – to + to indicate what has happened. Clicking on a + sign reverses the process.

This demonstrates that the browser contains a parser and the XML file has been parsed; its content can now be displayed as a tree structure.

ACTIVITY 2.2

Open the file `Non_XML.xml` using a text editor. You should see that it contains essentially the same XML as the `Simple_XML.xml` file except for one line, which has changed to a comment. Now open this file using a web browser. You will find that the browser reports an error – the file is not well-formed XML and so the parser cannot establish a proper tree structure.



ACTIVITY 2.3

Open the file `Simple_XML_rev.xml` using a text editor. You should see that it contains essentially the same XML as the `Simple_XML.xml` file, except for some additional text in the first paragraph of the first section. In particular, note that it contains entities for quotes. Now open this file using a web browser. You will find that the browser displays the parsed data content, replacing the entities with double quotation marks and removing any superfluous whitespace. Any indentation that you see is put there by the browser as a way of showing the tree structure.

ACTIVITY 2.4

Open the file `Internal_DTD.xml` using a text editor. You should see that it contains essentially the same XML as the `Simple_XML.xml` file, except for the addition of the internal representation of the DTD as shown in Figure 2.4. Now open this file using a web browser. You will find that the browser displays the elements of the document as previously, but without the DTD. There is no indication that the parser has used the DTD to validate the document.

ACTIVITY 2.5

Open the file `Bad_DTD.xml` using a text editor. You should see that it contains essentially the same XML as the `Internal_DTD.xml` file except for a change to the DTD, which means that the document is no longer valid according to the DTD. Now open this file using a web browser. You will find that the browser displays the elements of the document as in the previous activity, indicating that the parser is not using the DTD to validate the document.

In a similar way, a browser does not use an external DTD or a schema for validation, so we will not give examples.

EXERCISE 2.7

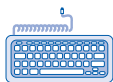
In Activity 2.5, what has been changed and what was the implication of this change that made the document invalid for the specified DTD?



ACTIVITY 2.6

Open the file `Simple_XSL.xml` using a text editor. You should see that it contains the stylesheet as shown in Figure 2.6, with the addition of an XML declaration and comment. Next open the file `Simple_XSL.xml` using a text editor. You can see that it is essentially the same as the file `Simple_XML.xml` except for the addition of a processing instruction for `xml-stylesheet`, which references the stylesheet file. Now open this file using a web browser. You should see the data content of the XML document displayed without any markup.





ACTIVITY 2.7

Open the file `Colour_XSL.xml` using a text editor. You should see that it is essentially the same as the file `Simple_XSL.xml` of the previous activity, except that the processing instruction for `xml-stylesheet` references the file `Colour_XSL.xsl` (you can examine the file if you wish, but it is not appropriate to explain the details). Now open `Colour_XSL.xml` using a web browser. You should see the data content of the XML document displayed without any markup, but it is presented differently from that in the previous activity.

Activities 2.6 and 2.7 demonstrate one of the main features of XML: it represents the logical structure of a document, which can then be presented in different ways according to the particular requirements of the medium or readers.

2.4 Summary

There are many concepts associated with XML. The term has come to mean a specified way of representing the logical structure and storage of a document, as well as a complex technology associated with the processing and presentation of documents and messages.

XML documents are represented in terms of elements and attributes using entities, where appropriate, with document definitions expressed as DTDs or schemas. XML documents may be viewed as a nested set of elements organised as a tree structure.

There are various forms of general processing of XML documents, including parsing, selection and transformation. These processes involve further XML specifications: XPath, XQuery and XSLT.

Finally, we must remind you that we have introduced only those aspects of XML that are necessary for consideration of XML and databases. There are many other features of XML, which we cannot possibly include in this topic. However, the overall message for you to note is that XML is different in many ways from the normal kind of data stored in databases, which is why it requires special treatment.

3

Using XML with databases

It is now time to consider the main purpose of this topic, which is the way XML can be used with databases, and specifically relational databases. We emphasise this point because there are DBMSs available that store XML directly but we are not considering them in this topic for two reasons. One reason is that most of the operational data in large organisations is held in relational databases, which determines the persistent form of the data that is to be represented in XML; this represents the main requirement for how XML can be used with databases. The second reason is that XML database management systems are still mainly in the early stages of development, with no standards apart from XQuery. Thus there is considerable variety in their approaches and it is not clear what we would be able to teach you that is generally applicable.

In this section, we first compare XML and relational data structures to show that there are significant differences between them, which justifies the special treatment for data being exchanged from one representation to another. Then we examine ways in which data from a relational database can be represented as XML, and conversely the ways in which the data in an XML document can be stored and queried in a relational database. This is presented in terms of the standard specification that is known as SQL/XML (formally it is ISO/IEC 9075 Part 14: XML-Related Specifications). It is a recent development of SQL, with the first edition of this part of the standard being published at the same time as the other parts of SQL in 2003, then a further edition being published in 2006. An SQL implementation may not support everything we describe, either because it has not yet implemented that capability or because it provides that capability in a non-standard way. We conclude with some practical examples of how SQL Anywhere supports XML.

3.1 Comparing XML and relational data

Block 2 of the course describes the relational concepts that provide the structure and manipulation for relational databases. The following summary focuses on the main features that are relevant to this comparison, expressed using SQL terminology.

- ▶ Relational data is held as atomic values in a tabular structure with a unique name.
- ▶ Columns of a table are identified by name within the table and all values in a column are of the same type; the order of columns is not significant.
- ▶ Rows of a table are distinct, distinguished by the values in each row (particularly the primary key); the order of rows is not significant.
- ▶ Access to and manipulation of data are expressed in terms of table operations that only involve value specifications (i.e. there is no concept of location of data by row number or column number).
- ▶ Relations are logical structures that do not have any storage implications.

In a similar way, we can summarise the main features of XML data as follows.

- ▶ XML data is held as nested elements in a tree structure with a named root element.
- ▶ Elements in a tree are named; they can have attributes with values and can contain other elements or character data, which are all represented as character strings, though an attribute or character data value can have a data type determined by a schema.

- ▶ An element is distinguished by its location in the tree structure, specified as a path in terms of named elements and sequence numbers from the root element.
- ▶ Access to and manipulation of elements and their contents are expressed in terms of operations that are based on the location of elements in the tree.
- ▶ XML is a logical structure with a specified storage representation as a sequence of characters.

It can be seen that there are significant differences in the way relational and XML data are structured and manipulated – on the one hand there is a collection of atomic values, each directly accessible, and on the other there are nested elements that are accessible via a path. There are many other detailed differences, but one distinction of interest for databases arises when there is no value; a table contains NULL for this purpose, whereas XML either just does not have an element (if it is allowed to be optional) or uses an empty element.

Because of these differences, there is no obvious transformation from one kind of structure to the other, such as there is for an entity–relationship model to a relational model. Thus the use of XML in relational databases needs to take account of these differences and provide ways in which the particular characteristics of XML data can be supported.



EXERCISE 3.1

Contrast the role of schemas for relational and XML data.

3.2 Transforming relational data into XML

We can distinguish two approaches to the general requirements for transforming data from a relational database into XML. One approach provides a way of outputting the data in a table into a predetermined XML structure that is independent of application requirements. The second approach provides special SQL capabilities that enable data from a relational database to create an XML structure that satisfies given application requirements, such as may be used for ebusiness messages.

Outputting data as predetermined XML structure

In this case, it helps to think of copying the data from a table into a file, where the file happens to contain XML. This is directly comparable to exporting data from a database in any other kind of format, which could be comma-separated values in a text file or some spreadsheet format. While we expect any database management system to provide export capabilities for various formats, this is not something for which you use SQL. (While some DBMSs include statements such as **EXPORT** or **OUTPUT** in their version of SQL, they are not specified as part of SQL.) Thus we should not expect SQL to provide a statement to output a table as XML, but what SQL/XML does specify is a mapping from a table to XML, or rather two mappings.

The first SQL/XML mapping produces an XML document having a root element whose name is the same as that of the table, and containing a sequence of elements named `row`. Each `row` element contains an element for each column, having the same name as that of the column and containing character data that is the value of that column in the corresponding row of the table. Figure 3.1 shows an example that consists of just three columns and two rows from the **staff** table of the University database.

```

<staff>
  <row>
    <staff_number>6451</staff_number>
    <name>Clark</name>
    <region_number>1</region_number>
  </row>
  <row>
    <staff_number>3678</staff_number>
    <name>Sanderson</name>
    <region_number>1</region_number>
  </row>
</staff>

```

Figure 3.1 An example of a mapping of a table to XML

The second SQL/XML mapping produces a sequence of elements, one for each row of the table. Each of these elements has a name that is the same as that of the table and contains an element for each column, having the same name as that of the column and containing character data that is the value of that column in the corresponding row of the table. Figure 3.2 shows an example of this mapping, containing the same data from the **staff** table as shown in Figure 3.1.

```

<staff>
  <staff_number>6451</staff_number>
  <name>Clark</name>
  <region_number>1</region_number>
</staff>
<staff>
  <staff_number>3678</staff_number>
  <name>Sanderson</name>
  <region_number>1</region_number>
</staff>

```

Figure 3.2 An example of an alternative mapping of a table to XML

In the example shown in Figure 3.2, there are two things to observe. First, it is not an XML document, because it does not have a single root element, but it is a sequence of well-formed XML elements. Such a sequence of elements is known as a **forest** (since it is a collection of trees). In practice, a database management system may add a root element to make it into an XML document so that it can be processed.

Secondly, the elements used to represent the data from the table are the same in both figures: each element has the same name as the column with character data from the table. If there is no value for a column – that is, it is null – this can be shown by an empty element with an attribute named `nil` to make it explicit, as in:

```

<region_number xsi:nil="true"/>

```

While these examples show the SQL/XML mappings of a table to XML, we must emphasise that they are not the only way to represent the data in a table as XML – they are just the ones specified by SQL/XML. A DBMS may choose to use a different mapping. You should also note that SQL/XML includes mappings for producing an XML schema for the XML document containing the table data.



EXERCISE 3.2

Explain why neither of the given mappings of a table to XML would be sufficient for copying a table from one database to another.

Creating an XML document for an application

In contrast to the SQL/XML mappings of a table to a pre-determined document structure that we have just described, what we will now consider is the creation of a document structure to satisfy given application requirements. For example, in describing the student transcript given in Figure 1.4 we mentioned that it was derived from the tables in the University database, so now we are going to address the question of how this can be done using an SQL/XML query. To see what is involved, we can give you the query that will produce the relevant XML as follows:

```
SELECT
  XMLELEMENT(NAME studentTranscript,
    XMLELEMENT(NAME studentId, s.student_id),
    XMLELEMENT(NAME studentName, name),
    XMLELEMENT(NAME enrolments,
      (SELECT XMLAGG(XMLELEMENT(NAME course,
        XMLFOREST(c.course_code AS courseCode, title, credit)))
      FROM enrolment e, course c
      WHERE s.student_id = e.student_id
        AND e.course_code = c.course_code)))
  AS transcript
FROM student s
WHERE s.student_id = 's05'
```

XML1

If you first want to be convinced that the query **XML1** does what we claim, you can jump to Activity 3.2 at the end of this section and see how to execute it and display the result.

The first point to notice about this query is that it has the structure you expect for SQL, with the outer part formed by the first line **SELECT** and the last two lines, **FROM ... WHERE ...**, specifying that data for student **s05** is required. There is also a subquery, again with the expected structure, specifying that data relating to this student's enrolments and courses is required. The rest of the query is a collection of functions, which together produce a single value, named by its concluding line as **transcript**. Let us now examine these functions one at a time so that we can see how they produce the required XML result.

The functions we are considering are referred to either as XML constructor functions, because they are used for constructing XML, or as XML publishing functions, because the results are often destined for publishing on a website.

The most important function is **XMLELEMENT**, which has a first argument that is the name of the element it produces (prefaced by the required **NAME**) followed by values of the content of that element – that is, either further elements or character data. It is also possible to include an argument for attributes of the element (using the function **XMLATTRIBUTES**) but we are not going to consider such details here. Let us look at a simple example of using **XMLELEMENT** in the following query, which involves the **staff** table of the University database.


```
SELECT staff_number,
       XMLELEMENT(NAME staffName, name)
       AS XML_name
FROM staff
WHERE region_number = '1'
```

XML2

In this example, for each row of the result, `XMLELEMENT` will produce an XML element with the name `staffName` and containing character data taken from the `name` column of the `staff` table, as shown in the following table:

staff_number	XML_name
3678	<staffName>Sanderson</staffName>
6451	<staffName>Clark</staffName>

As a slightly more complex example, the following query shows how to produce elements containing other elements – that is, nested elements.

```
SELECT staff_number,
       XMLELEMENT(NAME staff,
                  XMLELEMENT(NAME staffName, name),
                  XMLELEMENT(NAME staffRegion, region_number))
       AS XML_staff
FROM staff
WHERE region_number = '1'
```

XML3

In this example, you need to see that for the three lines starting with `XMLELEMENT`, the second and third lines are contained within the first one. We have used indentation to help make it clear, but you cannot always expect this and you need to examine where the parentheses are placed. In this case, the closing parenthesis for the first `XMLELEMENT` appears after the third `XMLELEMENT`. This query gives the following result.

staff_number	XML_staff
3678	<staff> <staffName>Sanderson</staffName> <staffRegion>1</staffRegion> </staff>
6451	<staff> <staffName>Clark</staffName> <staffRegion>1</staffRegion> </staff>

One point to note about this table is that we have shown the XML values in a (hopefully) recognisable form so that you can understand what is produced, but we shall consider the actual form of the result of `XMLELEMENT` in Subsection 3.4.

You should now be able to appreciate the first few lines of query **XML1**. The first `XMLELEMENT` function produces the element named `studentTranscript` and it contains three more `XMLELEMENT` functions producing the elements `studentId`, `studentName` and `enrolments`, which, if you refer to Figure 1.4, is the required structure. In Figure 1.4, the content of the `enrolments` element is a sequence of course elements, each containing `courseCode`, `title` and `credit` elements. This is now considered in the following text.

The content of the `enrolments` element is produced by the subquery in **XML1**, using the function `XMLAGG`. This is an aggregating function that works in the same way as the SQL functions such as `SUM` and `AVG`. Where a query includes `GROUP BY` to partition the rows of a table into groups, or to treat a complete table as a group, an aggregating function produces a single value from the collection of values taken from each group of rows. `XMLAGG` works on an XML expression and produces a result, which is a sequence of elements formed from the concatenation of the XML values from each group. The following query gives a simple example of its use.

```
SELECT XMLAGG(XMLELEMENT(NAME sName, name) )
      AS staff_names
FROM staff
WHERE region_number = '1'
```

XML4

The result of this query is a single value (actually a table with a single row and single column) formed by concatenating the two XML elements created for the two rows selected in this query, as follows:

```
<sName>Sanderson</sName><sName>Clark</sName>
```

In the query **XML1**, the subquery that includes the function `XMLAGG` is used to produce the sequence of `course` elements, so now we can look at how the content of each `course` element is specified. It involves yet another function, `XMLFOREST`, that also produces a sequence of elements like `XMLAGG`, but in this case the sequence is derived from a list of arguments, each of which can be any expression whose value is used as the character content of an element. An element name can be explicitly specified using the usual `AS` notation, but if an argument is just a column name then the element name is implicitly the same. A simple example of the use of `XMLFOREST` is given in the following query:

```
SELECT staff_number,
      XMLFOREST(name, region_number AS region)
      AS XML_details
FROM staff
WHERE region_number = '1'
```

XML5

The result of this query is shown in the following table:

staff_number	XML_details
3678	<name>Sanderson</name><region>1</region>
6451	<name>Clark</name><region>1</region>

You might notice that this result is very similar to the result for query **XML3**, and that query could have been written using `XMLFOREST` instead of having the second and third `XMLELEMENT` functions to produce the required sequence of elements.

Finally, returning to the query **XML1**, `XMLFOREST` is used to give the sequence of elements `courseCode` (the assigned element name for values from column `c.course_code`), `title` and `credit` for a particular course, which provides the content for a `course` element. As already described, all `course` elements (just two in our example) are combined by the `XMLAGG` function to provide the content for the `enrolments` element.

This concludes the explanation of how the query **XML1** gives the XML in Figure 1.4, which has demonstrated the use of the three main SQL/XML functions. There are other publishing functions, but we are not considering any more details since our purpose is to show the principles of how XML data can be constructed from relational data.

EXERCISE 3.3

Describe the main reason why you would choose to use the SQL/XML publishing functions to create an XML document, as opposed to using a predetermined mapping to output a table as XML.



3.3 Storing XML data in a relational database

While the previous section examined how relational data could be transformed into XML, in this section we are considering the converse: how XML data can be stored in a relational database. There are two distinct approaches: one that uses the SQL data types with which you are already familiar, and another that uses the XML data type introduced by SQL/XML.

Storing XML in tables

We first consider how XML may be stored in a database by extracting data from an XML document and storing that data in the tables of a database. One way of doing this decomposes an XML document into simple values that can be represented in tables; this is known as **shredding** a document. In the shredding of a document, you need to consider how much of the document is to be stored and the extent to which the original XML document can be regenerated from its tabular representation. While it is simple to record the value of any character data as just a character string, what is essential is to represent the structure of each document – a tree.

Let us show you what this involves with an example. Suppose the recipient of the transcript XML document given in Figure 1.4 receives many such transcripts and wants to store the data in a database. The first issue is to decide what tables need to be used for this purpose. It is possible to have just one table that can store the data from any XML document, which means that element names have to be treated as data in the table. This can be done by assigning an identifier to each node in the tree and representing the structure by referencing its parent node. One simplified way of doing this is given in the following table, which gives a representation of the transcript document from Figure 1.4.

node_id	node_name	parent_id	content
1	studentTranscript		
2	studentId	1	s05
3	studentName	1	Ellis
4	enrolments	1	
5	course	4	
6	courseCode	5	c2
7	title	5	Syntax
8	credit	5	30
9	course	4	
10	courseCode	9	c7
11	title	9	Pragmatics
12	credit	9	30

As you should expect, the table has the same number of rows as the nodes of the tree structure for the transcript document given in Figure 2.1. This kind of representation of an XML document is known as an **edge table** because it is representing the connections from each node to its parent – these are referred to as the edges in a graph. An edge table can also include attributes of elements by treating them as further nodes in the tree. While such a table can represent any XML document, to include many documents would require a more general table with some means of identifying each document.



EXERCISE 3.4

Explain why it is important that we have chosen a sequential numbering for the node identifiers in this table.

A different approach to shredding an XML document requires tables that more closely correspond to the structure and content of the document. For the transcript document, the data could be stored in tables that are the same as those in the University database from which the data originated, but they don't have to be the same. Indeed, the recipient organisation is not likely to know anything about the University database. For example, the following tables show how the data content of the transcript in Figure 1.4 can be recorded in a slightly different way from that used for the same data in the University database.

Note that the `xml` in this case is just part of the table name, which is used to distinguish it from the tables in the University database.

student_xml

student_id	student_name
s05	Ellis

enrolment_xml

student_id	course_code	title	credit
s05	c2	Syntax	30
s05	c7	Pragmatics	30

In this case, the XML document given in Figure 1.4 results in one row for the `student_xml` table and two rows for the `enrolment_xml` table, with values in each column given by the character data contained in each element. Any other transcript can be decomposed in the same way to give additional rows for these tables.

However, this is not the only way to record data from the transcript XML document. In general you can think of it as a problem of designing a database, as described in *Block 4*, where the XML documents to be stored, and any associated DTD or schema, are only part of the requirements to be considered in determining what tables are appropriate. Since this topic focuses on XML, we are not considering other possible requirements and their impact on designing tables to receive the XML data. What we do need to consider, because there can be several tables required for each XML document, is the kind of process that can transform the XML data into values for the corresponding columns in the tables.

You could develop some special-purpose application process that takes an XML document, extracts the relevant parts from it and stores them in given database tables. The disadvantage of this approach is that you will need to develop a separate application process for each of the XML documents whose data you want to store (or, more generally, for each DTD or schema defining those documents).

A better approach is to use general-purpose shredder software that can transform any XML document into tables. Since it is general-purpose, such shredder software requires a specification which gives details of how a given XML document is to be transformed into tables. This is provided by an XML to table mapping that states how each part of the XML document is to be stored in a table. There are many different implementations of this approach, but we do not wish to give details of any particular one. However, they all work on the same principle and this is shown in Figure 3.3.

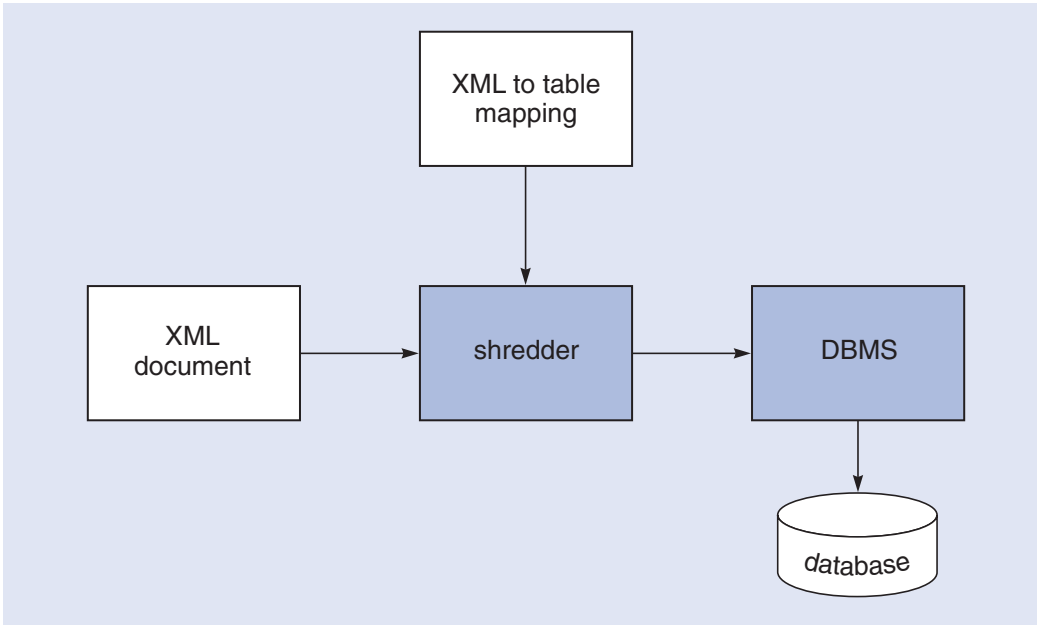


Figure 3.3 Shredding an XML document

SQL Anywhere does not support the functions in this section, so do not attempt to execute the following statements. In the activities, we examine how SQL Anywhere provides similar capabilities.

In Figure 3.3, we have shown the shredder process as separate from the DBMS in order to clarify its input and its output, which is stored in an ordinary database. In practice, you may often find that a shredder is packaged as part of a DBMS and it is only possible to use it with that DBMS. You should also note that the ‘XML to table mapping’ is often an XML document itself.

Now let us consider an alternative way of storing XML data in tables, which is different from shredding because it is concerned with extracting particular values from an XML document for inserting into the columns of a single table. SQL/XML specifies a function `XMLTABLE` that can be used for this purpose.

The function `XMLTABLE` takes as an argument an XQuery expression given as a character string, which results in some XML, and a list of columns, each with an XPath expression that is applied to the XML to give the value for that column in the resulting table. Note that because the result of the function is a table, it is used in a `FROM` clause. As an example, we can insert the student data from the file `Transcript.xml` into the `student_xml` table by using the simple form of XQuery to give the XML element `studentTranscript`, from which the given paths assign values to the columns `student_id` and `student_name`, as follows:

```
INSERT INTO student_xml
SELECT *
FROM
XMLTABLE('doc("Transcript.xml")/studentTranscript'
COLUMNS
student_id CHAR(3) PATH 'studentId'
student_name VARCHAR(15) PATH 'studentName')
```

XML6

This example is provided only to demonstrate what is possible, and we are not giving any further details on how to use this function. However, we have provided another example of its use in the next subsection.



EXERCISE 3.5

Suppose you have a database with tables designed for many existing application processes, and you receive an XML document (or many documents having the same DTD or schema) containing data to be stored in those tables. Briefly explain what options you have and what needs to be done in each case.

Storing XML as a value

We have described how an XML document is just a character string. We will now consider treating the whole document as a single value that can be stored in a column of a table, rather than decomposing it in some way as described in the previous subsection. For example, instead of the `staff` table as defined in the University database, a table `staff_xml` can be defined as follows:

```
CREATE TABLE staff_xml (
staff_no CHAR(4),
staff_details VARCHAR(250),
PRIMARY KEY (staff_no))
```

XML7

For this table, XML data can be stored by an **INSERT** statement such as:

```
INSERT INTO staff_xml
VALUES ('8247',
      '<staff>
        <name>Shaw</name>
        <region>2</region>
      </staff>')
```

XML8

When storing XML in this way, a DBMS accepts whatever character string is provided by the XML application that performs the insertion, without any recognition or checking that it is XML (adding an XML declaration, `<?xml ... ?>`, does not help). Another XML application that retrieves this value receives a character string, and the application may then need to parse the character string to check that it is well-formed XML or is valid according to some DTD or schema. In this case, you are not really storing XML in a database; you are storing character strings and it is the XML applications that treat the character strings as XML. This approach is appropriate for those XML applications that do not want a DBMS to do anything other than just store and retrieve complete XML documents.

To enable a DBMS to recognise that it is storing XML and help it to support XML processing, SQL/XML specifies a new data type called **XML** whose values are parsed XML. The **XML** data type has several subcategories; it is possible to specify whether a value is an XML document, which must be well formed with a single root element, or some other less restrictive form of XML that does not have a root element, such as a forest. It is also possible to specify whether the values are valid according to an XML schema. We use the term **XML value** to refer to any instance of the **XML** data type.

For example, the **staff_xml** table can be defined to allow the column **staff_details** to contain XML values of some implicit category, as follows:

```
CREATE TABLE staff_xml (
  staff_no CHAR(4),
  staff_details XML,
  PRIMARY KEY (staff_no))
```

XML9

To insert an XML value into this table requires that its character string representation, such as that given for **XML8**, is parsed to ensure that it is acceptable XML. There is a function, **XMLPARSE**, for this purpose that also requires you to choose how whitespace is to be treated during the parsing, as given in the following example:

```
INSERT INTO staff_xml
VALUES ('8247', XMLPARSE(
  '<staff>
    <name>Shaw</name>
    <region>2</region>
  </staff>'
  STRIP WHITESPACE))
```

XML10

In this example, the whitespace is removed as in normal XML parsing. The alternative syntax is **PRESERVE WHITESPACE**, which is required in some XML applications.

Having inserted an XML value using **XMLPARSE**, you might ask what is actually stored in the database. However, this is not a question we can answer because, just as SQL does not specify how a DBMS stores a number – only what you can do with it – a DBMS may store an XML value in different ways. In describing parsing, we have explained how an internal tree structure can be created, and you can still think of it that

way to understand how it can be processed. However, a DBMS may use any kind of structure to store its XML values since it is only for use by the DBMS. This includes tables and indexes that you expect to find in relational databases, but it can be any kind of storage mechanism, in particular, one that is specially organised for efficient processing of XML. At the other extreme, it is also possible to store just the parsed character string, though that would not be very efficient for subsequent XML processing. But even in this case, it is important to remember that such an XML value cannot be used as a character string – it is now a value of type **XML** and, therefore, has the properties of that data type.

So, what are the properties of an XML value? What can you do with it? Well, you cannot do much with an XML value by itself. As you have just seen, the XML value produced by **XMLPARSE** can be inserted into a column. You cannot compare two XML values – there is no consistent answer in asking if one value is equal to another, never mind whether one is greater than another. However, there are some functions that operate on XML values.

For example, one function that is directly comparable to the concatenation of character strings is provided by **XMLCONCAT**, which can have two or more XML values as arguments and produces a single XML value that combines the elements in these values in sequence. We shall consider other functions concerned with querying XML values in the next subsection.



EXERCISE 3.6

Explain why the result of the **XMLCONCAT** function is not a well-formed XML document.

3.4 Querying XML values in a relational database

In the previous subsection, we showed how you can store XML documents in a table as a value of the XML data type, and now we will consider how such values may be queried. There are two distinct approaches to this querying: one is based on relational concepts and the other is based on XML concepts.

Relational approach

For the relational approach, we need to be able to process XML values so that they can then be treated like any other relational data in an SQL query. But we have already described a function that transforms an XML value into a table: **XMLTABLE**. The following query shows how this function can be used for the XML values in the column **staff_details** in the table **staff_xml**. In this example, the **PASSING** clause specifies the column containing the XML value that is to be used by the function:

```
SELECT staff_no, staff_name
FROM staff_xml s,
XMLTABLE('$d/staff'
PASSING s.staff_details AS "d"
COLUMNS
staff_name VARCHAR(15) PATH 'name'
staff_region CHAR(2) PATH 'region')
AS details_table
WHERE staff_region = '2'
```

In this query, for each row of the table `staff_xml` the XML value in the column `staff_details` is referenced as variable `d` as part of the argument for the `XMLTABLE` function. This is used in the simple form of XQuery '`$d/staff`' to provide the context from which the columns `staff_name` and `staff_region` are extracted by using the given paths. In this case, `XMLTABLE` produces only a single row to be joined with each row of `staff_xml`, but in general there could be many rows.

This example demonstrates how `XMLTABLE` can be used to transform an XML value into a tabular form that can be included in an SQL query, and thus make use of the full power of SQL to search and retrieve data from XML values. In addition, you should note that the use of XQuery, which is trivial in our example, can be much more complex so that it can also be used as part of the search process by applying conditions on the XML values.

Another aspect of this example to note is that the result of `XMLTABLE` does not contain XML values, so that the rest of the query and the result of the query does not have to be concerned with XML.

XML approach

We now consider the XML approach to querying, where the emphasis is on searching the XML values rather than just converting them into a table. By this we mean an approach similar to the way character strings in a table can be queried, in terms of both identifying particular character strings that match a given pattern, using the `LIKE` predicate (see Section 2 of *Block 3*), and extracting parts of character strings using functions such as `SUBSTRING`. But XML already has its own query language – XQuery. Rather than define another way of querying XML, SQL/XML provides a function called `XMLQUERY` that allows you to give an XQuery expression that can be applied to values of type `XML` in a column of a table.

As a simple example, the following SQL query uses this function by applying the XQuery expression specified as a character string to each XML value in the column referenced by the `PASSING` clause (all given as a single argument of the `XMLQUERY` function):

```
SELECT XMLQUERY('for $d
  where $d/staff/region = "1"
  return $d/staff/name'
  PASSING staff_details AS "d"
  RETURNING NULL ON EMPTY)
  AS XML_name
FROM staff_xml
```

XML12

The function `XMLQUERY` returns an XML value for each row of the table `staff_xml`. The two main parts of the argument of the function are the XQuery expression given as a character string, '`for...` ', and the `PASSING` clause, which specifies that values from the column `staff_details` are to be passed to the XQuery expression using the variable `d`. The resulting table is determined by the result of the XQuery expression, which returns an XML value containing the element `name` from each value in the column `staff_details` where the element `region` contains a value 1. For those rows that do not satisfy this condition, there is no result from the XQuery expression – it is an empty sequence – in which case, the additional `RETURNING` clause specifies that `NULL` should be returned.

In this example, note that we have explicitly described the result as being an XML value – that is, the function returns a value of type `XML`. When describing the functions `XMLELEMENT`, `XMLAGG` and `XMLFOREST`, we have avoided the issue and showed the

In this example, if you did not want an XML value returned, you could append `/text()` to the `return` clause of the XQuery (see the description of XQuery in Subsection 2.2).

result of the example queries in Subsection 3.2 as character strings, whereas they all produce a result that is an XML value. So why is it an issue?

We have emphasised that an XML value is not just a character string, and it is important to know what type you are dealing with because it determines what you can do with it. For example, you cannot use the **LIKE** predicate with an XML value. In situations when you do need a character string representation of an XML value, SQL/XML provides the function **XMLSERIALIZE** for this purpose. This function takes an XML value as an argument and converts it to a character string, as shown in the following example:

```
SELECT XMLSERIALIZE(staff_details AS VARCHAR(250))
       AS xml_details
FROM staff_xml
```

XML13

In this example, the result produced by **XMLSERIALIZE** is the same character string representation as used for inserting values into the column **staff_details** in **XML10**, without whitespace but keeping the same sequence of XML elements.

Fortunately, you do not have to use **XMLSERIALIZE** explicitly in every query because for XML applications and query tools an XML value can be automatically transformed into a character string for processing by the application or display by the tool.



EXERCISE 3.7

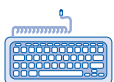
The functions **XMLTABLE** and **XMLQUERY** have a common mechanism for specifying that an XQuery expression is to be applied to a column of type **XML**. Describe how that mechanism is specified.

We are not providing any more details of querying XML data. It is enough that you appreciate the two distinct approaches: one that is based on transforming XML values to a tabular form for use in a relational query, and the other that relies more on the power of XQuery to select and retrieve from the XML values to provide an XML result.

3.5 Activities

The activities provided for this section are intended to provide demonstrations of some of the ways in which XML can be used with the course software, SQL Anywhere. In some cases we can show how it supports the SQL/XML capabilities as we have described, and in others we show how it provides an alternative way of providing similar capabilities.

In these activities, you are expected to be able to examine XML files as described in the activities in Section 2. They are all based on using SQL Anywhere connected to the University database.



ACTIVITY 3.1

This activity demonstrates how SQL Anywhere provides the capability of outputting a table as XML. As described in Subsection 3.2, this is commonly provided as a way of exporting data from a database, and in SQL Anywhere this is performed on a table that is the result of a query, as displayed in the Results panel of the Interactive SQL tool. You have the choice of using the Export wizard (look under the ISQL Data menu) or executing an **OUTPUT** command. In this activity, we use the **OUTPUT** command to be clear about what is required. The following two statements produce an XML document

containing the data from the `course` table. The first statement retrieves the required data and the second transforms it into XML and stores it in the specified file. The file path given is that recommended for the course activities, but you can change it to whatever you like as long as the folder that you specify already exists. Note that the given file path requires double quotation marks because it includes spaces.

```
SELECT * FROM course;
OUTPUT TO "C:\Program Files\M359\Activities\Block 5\course_output.xml"
FORMAT XML;
```

Execute these statements, separately or in combination, and then check to see that the file has been created. If you open the file with Notepad, it will appear as an unformatted sequence that is difficult to read, but it is more readable if you use WordPad because it recognises the end-of-line character being used. One feature you should be able to recognise is that it contains a DTD. Open the file with a browser and you can see that its structure is different from either of the mappings described in the course text.

EXERCISE 3.8

Give a description, in general terms applicable to any table, of the way SQL Anywhere represents a table as an XML document.



ACTIVITY 3.2

This activity is concerned with demonstrating the SQL/XML publishing functions as used in the queries **XML1** to **XML5**. You can choose to enter these queries yourself, exactly as given in the course text, or you can execute them from the file `Transcript.sql` that we have provided for you.

In that file we have provided the queries in the sequence **XML2** to **XML5**, the order in which they are explained in the course text, and the last being **XML1** – the complete query that combines the functions to produce the transcript. Each query produces a result in which the XML is displayed as a character string without any formatting.

For the last query, **XML1**, you might like to check the XML it produces by exporting the result as follows (the `OUTPUT` statement is included in the `Transcript.sql` file, and our previous comment about the path for the output file also applies):

```
OUTPUT TO "C:\Program Files\M359\Activities\Block 5\transcript_output.xml"
FORMAT ASCII
```

In this case, because we have already generated the XML as a character string, we specify that the result is put into the file using ASCII coding. If you open the file with a text editor, you will find that it is an unformatted character string. However, having been output as a character string, it will have single quotation marks on both sides of it. Use your text editor to remove the single quotation marks at the beginning and end of the file. Then save the file and reopen it with a browser and compare the result with the original presentation of the transcript XML document in Figure 1.4.



ACTIVITY 3.3

This activity shows how SQL Anywhere provides an equivalent capability to the SQL/XML function `XMLTABLE` described above. The SQL Anywhere function is called `OPENXML` and it extracts data from an XML document in a similar way to `XMLTABLE`, though with slightly different syntax. Shown below is an example of using `OPENXML` in an `INSERT` statement to perform the same task as the statement **XML6**, though in this case

you can see that the transcript XML document is embedded within the statement rather than using a file reference as in **XML6**. The XPath specification of the context in the document, `/studentTranscript`, is given as a separate argument (following the end-tag `</studentTranscript>`) rather than as part of an XQuery expression as in **XML6**, and the columns for the resulting table are specified in a separate **WITH** clause rather than using **COLUMNS** embedded as part of the argument for **XMLTABLE**.

```
INSERT INTO student_xml
SELECT *
FROM OPENXML(
    '<studentTranscript>
    <studentId>s05</studentId>
    <studentName>Ellis</studentName>
    <enrolments>
    <course>
    <courseCode>c2</courseCode>
    <title>Syntax</title>
    <credit>30</credit>
    </course>
    <course>
    <courseCode>c7</courseCode>
    <title>Pragmatics</title>
    <credit>30</credit>
    </course>
    </enrolments>
    </studentTranscript>',
    '/studentTranscript')
WITH(
    student_id CHAR(3) 'studentId',
    student_name VARCHAR(15) 'studentName')
```

This **INSERT** statement is provided in a file named `Transcript_openxml.sql`. Open this file with a text editor to check that it is the same as that given here. Use ISQL to create the table `student_xml` (this statement is included in the file `Transcript_openxml.sql`) and then execute the given **INSERT** statement. On querying the `student_xml` table, you should find that it contains the row shown for that table in Subsection 3.3, which is also the result described for **XML6**.

ACTIVITY 3.4

This activity examines the extent to which SQL Anywhere supports the **XML** data type. The statements for this activity are provided in the file `Staff_xml.sql`.

To begin with, open this file in ISQL so that you can execute statement **XML9** to show that the **XML** data type is accepted as a valid data type for a column.

Having defined the table `staff_xml`, next execute the **INSERT** statement **XML8** given in the file, and then execute the **SELECT** statement to display the inserted XML. Notice that we have asked you to execute **XML8** and not **XML10**, which is the **INSERT** statement that includes the **XMLPARSE** function. So, what is SQL Anywhere doing if it allows an unparsed XML character string to be inserted as a value of its **XML** data type? A simple check is to see whether you can insert a character string that is not well-formed XML. The file `Staff_xml.sql` contains an example **INSERT** statement for you to try. You will find that any character string can be inserted, so you can conclude that the SQL Anywhere **XML** data type is just an unparsed character string.

The file `Staff_xml.sql` contains a **DELETE** statement so you can remove the row containing the non-XML character string before continuing.

You can find that **XML** is actually defined as a domain, but it is still used in the same way as a data type in a **CREATE TABLE** statement.

Even though it is not parsed, an XML character string has been inserted into the `staff_xml` table, and that XML value can be queried using `OPENXML` in a similar way to the use of `XMLTABLE` in the query `XML11`. You will find the following statement in the file `Staff_xml.sql`, which you can compare with `XML11` in the same way that we have described for the use of `OPENXML` in Activity 3.3.

```
SELECT staff_no, staff_name
FROM staff_xml s, LATERAL (OPENXML(s.staff_details, '/staff')
    WITH (staff_name VARCHAR(15) 'name',
        staff_region CHAR(2) 'region'))
AS details_table
WHERE staff_region = '2'
```

Execute this query to show that it extracts the required data from the XML value. You can insert more rows of data into the `staff_xml` table if you want to have a more effective test of this query (as long as you ensure that the values you insert into the column `staff_details` are well-formed XML).

You should note that the keyword `LATERAL` is a part of SQL syntax required when a function or subquery in a `FROM` clause references a previous table; this is implicit in `XMLTABLE` but is required for `OPENXML`.

3.6 Summary

XML can be used in so many ways that it has led to various kinds of capability to support XML applications that involve databases.

At the simplest level, a database may not contain any XML data but there can be a need for data from the database to be provided as an XML document. This can be supported either by exporting tables in XML format or by using the publishing, or constructor, functions of SQL/XML to present the results of the queries as XML.

When there is a need to store data that is represented as XML, one possibility is to transform it – shred it – so that it becomes a collection of data in otherwise normal tables. Alternatively, the XML can remain as a single value, either stored as a character string or parsed and stored as a value of the `XML` data type.

When XML is stored in a database, its content can be queried either by transforming it so that it can participate as a table in an SQL query, or by using XQuery – the XML query language – to provide the required result.

With so many choices available, it is essential that the requirements of the XML applications are carefully taken into account in deciding both the design of the database and the way the data is going to be processed. However, you do need to appreciate that producing XML is usually just the first step of an XML application. For example, XML cannot be displayed with all its markup but needs a stylesheet in order to transform it into a readable format for presentation.

Topic summary

This topic is concerned with the use of XML as a means of representing data and the special treatment required when that data is stored in or retrieved from a database.

Overall, the message of this topic is that XML has become very important to many areas of computing and it is a complex subject in its own right. So, when it is used in combination with relational databases, there are many difficulties in identifying and evaluating the possible options and in implementing the most appropriate technology. It is hoped that you now recognise the numerous issues involved.

LEARNING OUTCOMES

After studying this topic, you should be able to:

- ▶ Describe the essential concepts of XML and its processing.
- ▶ Explain why XML requires special treatment in relational databases.
- ▶ Identify the differences between the relational view of data and the XML view of data.
- ▶ Describe the alternative ways in which relational data can be output as XML.
- ▶ Describe the alternative ways in which XML may be stored using relational tables.
- ▶ Describe the use of the **XML** data type and how it may be queried.

Solutions to Topic 1 Exercises

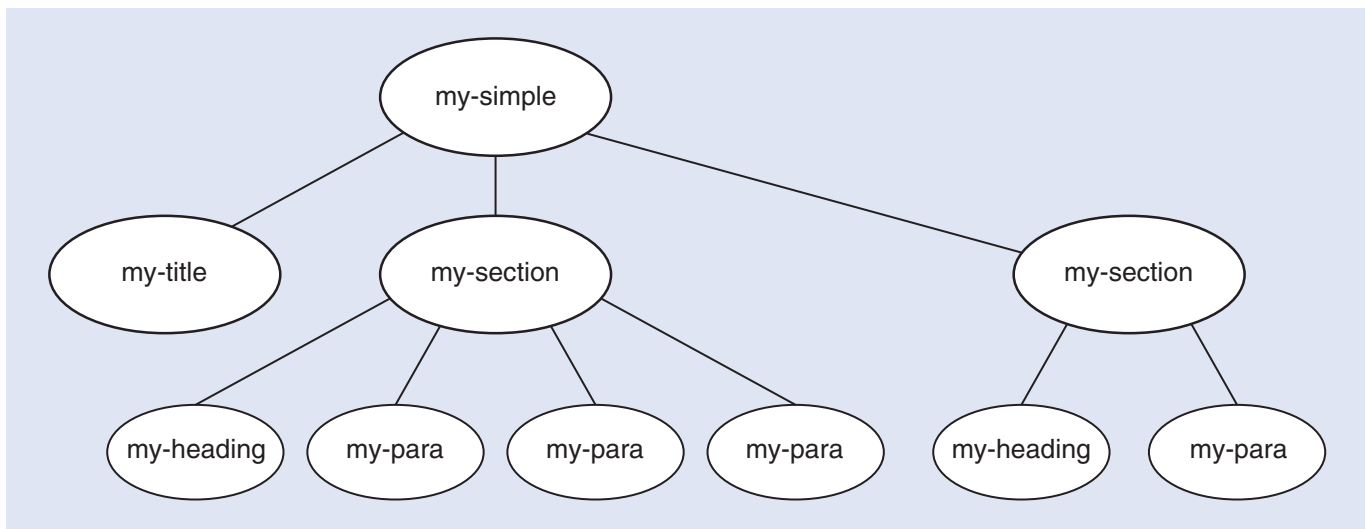
SOLUTION 2.1

They are studentId s05, studentName Ellis, courseCode c2, title Syntax, credit 30, courseCode c7, title Pragmatics, credit 30.

SOLUTION 2.2

The given text does not have a root element. One can be created by adding a start-tag, say `<my-simple>`, before the given text and an end-tag, `</my-simple>`, after it.

SOLUTION 2.3



SOLUTION 2.4

They are studentTranscript, studentId s05, studentName Ellis, enrolments, course, courseCode c2, title Syntax, credit 30, course, course, courseCode c7, title Pragmatics, credit 30, course, enrolments, studentTranscript.

SOLUTION 2.5

```
<studentTranscript source="Walton University" date="2006-10-29">
```

Note that the date is just another character string, but some acceptable format needs to be used. In this solution we have written it according to the international date format.

SOLUTION 2.6

```
<!ELEMENT studentTranscript (studentId, studentName, course*)>
```

SOLUTION 2.7

The `*` was removed from `my-para` in the specification of the element `my-section`, which means that only one `my-para` element is allowed in each `my-section` element, whereas the first section contains three paragraphs.

SOLUTION 3.1

Relational data cannot exist without a schema since a DBMS requires the schema definitions to manage anything it does with that data.

XML data can exist without any schema. The role of a schema is mainly to check whether an XML document is valid according to such a definition, though one can also be used to control what is allowed in creating an XML document (to ensure that the result is valid).

SOLUTION 3.2

While the resulting XML contains the rows of data for a table, it does not provide any details of the data types and possible constraints needed to create a table.

SOLUTION 3.3

The main reason is to create an XML document that is structured to satisfy particular requirements, which in an ebusiness environment may be imposed by other organisations.

SOLUTION 3.4

A relational table has no ordering. So, in order to represent fully the XML document (and be able to regenerate it from this tabular representation), the sequential nature of the original document must be explicitly captured.

SOLUTION 3.5

One option is to use the **XMLTABLE** function, in which case you need to specify how the data for each table is to be extracted from the XML document using XQuery.

Another option is to shred the XML document using a mapping that determines how the character data for each element (and possible attributes) of the XML document is to be transferred to a column in some table in the database.

A further possibility is to shred the XML document as an edge table, and then use normal SQL queries to extract data from the edge table for insertion into the required tables.

SOLUTION 3.6

Even if the arguments for the **XMLCONCAT** function are themselves well formed, the combination of them cannot have a single root node, and so the result cannot be a well-formed XML document.

SOLUTION 3.7

The mechanism for specifying that an XQuery expression is to be applied to a column of type **XML** is given by a **PASSING** clause, which names the column and associates it with a variable that is used within the XQuery expression.

SOLUTION 3.8

The XML document has a root element named `resultset` that contains a sequence of elements named `row`, one for each row of the result table. Each `row` element contains a sequence of elements named `column`, one for each column of the table. Each `column` element has an attribute named `name` whose value is the name of a column and an attribute `null` having a value that is either `true`, in which case it is an `empty` element, or `false`, in which case the element contains character data that is the value of the column from the result table.

Topic 2 Application development

1 Introduction

This topic is about some of the various approaches to developing application processes and the different ways in which an application process can access a database. This topic is included in the course because, in practice, most access to the kind of databases used within organisations is by means of application processes. We do not want to leave you with the impression that everything can be done using database tools such as those provided for you in this course.

An application process was introduced in Subsection 3.8 of *Block 1* as one kind of user process. The other kind of user process was referred to as a database tool, a term that encompasses a range of general-purpose software providing facilities such as querying, reporting and managing databases. Apart from some examples of application processes in *Block 1*, you have been using the database tools, Sybase Central and ISQL, in your practical activities. These tools enable you to see tables and their properties as well as manipulating them directly by entering SQL statements, without having to be concerned with any other aspect of using a database.

We are now going to examine a rather more complex view of accessing databases that is required when developing application processes. As opposed to the general-purpose nature of database tools, an application process is usually designed to provide access to a particular database for a specific purpose, using predetermined tables and incorporating a user interface appropriate to the application requirements of its users. An application process is prepared either by using a programming language to write a program or by using an application development tool that enables more rapid production for some purposes. There are various kinds of application development tools with different approaches, many of which have the aim of hiding the complexity of a database from the developer. However, in order to provide you with a more detailed understanding of what is involved in application development, we are focusing on the use of programming languages and aim to establish general principles that are applicable to application development tools.

We are not assuming that you are familiar with any particular programming language. So we do not aim to teach you how to produce complete programs and, in particular, we are not considering how data might be manipulated within a program, how the results of that processing might be displayed or how exceptions are handled. Thus we are focusing on the database access mechanisms required for application processes and, where possible, we aim to establish the concepts common to different programming environments. However, Java is the programming language that has been widely adopted for database development and it is the main language taught within the OU Computing programme, therefore database access using that language is considered in detail.

1.1 The need for application processes

Your familiarity with using database tools might lead you to think they provide all the capabilities necessary for accessing and manipulating data in a database, so first we consider why application processes are needed. This may be summarised as both controlling and helping users.

There is a need to control users so that they can only access data to which they are entitled and, in particular, they can only change data for which they have been authorised to do so. Controlling changes involves not only restricting the types of data that can be accessed, but also checking the values that are entered. While SQL provides various facilities for this kind of control, including privileges on tables and check constraints on the values allowed in columns, it is a design choice as to how much of this control is implemented in a database and how much is enforced within application processes. An application process can be programmed to provide a much more detailed level of control than can be defined using SQL facilities, such as checking where a user is located, or what time it is, before allowing changes to be made to a database.

Just as important, there is a need to help users access a database so that they do not need to know how data is stored, nor how to specify what they want using a language like SQL. Furthermore, an application process should be able either to hide completely from the user or to explain comprehensibly the effects of different kinds of error and exception conditions, such as those that arise from constraint violation and transaction failure.

Apart from helping users access a database, an application process should also help them to understand the data and its effectiveness when it is used. This is likely to involve presenting results in a way that corresponds more closely to a user's organisational context, such as displaying data in a format other than tables, as well as providing additional descriptive text to explain the meaning of results.

1.2 Varieties of application process

You need to appreciate that there is a range of different kinds of application process and approaches to development that are possible, but in this topic we are not able to consider all of them in the same level of detail.

Thus we briefly describe the main features of database access for the traditional kind of programming language, such as C, COBOL and Fortran, and then we examine in more detail the facilities provided for the language Java by JDBC (Java Database Connectivity). Other ways in which Java can be used for database access are described in order to establish the alternatives that are possible. Finally, we provide a short introduction to the use of scripting languages, which are widely used in a web-based environment, so that you are aware of further options. The overall aim is to establish an understanding of principles of database access that can be applied in different contexts.

Apart from the choice of programming language, there are many other issues involved in the design of application processes and how they interact with databases. There are some basic principles, such as selecting the required data precisely so that you do not retrieve a complete table when only a few rows are required, but many other aspects of application design depend on the environment in which it is to be used. This becomes a particular problem in a networked environment when there are many factors, such as the use of stored functions and procedures (described in Section 7 of *Block 3*), that need to be considered. However, in order to focus on mechanisms for database access, we are not considering these aspects of designing application processes. For most of this topic, we will assume that application processes execute in the same computer system as the database management system.

1.3 The rest of this topic

Having established the scope of what we include in this topic, the remaining sections are presented mainly in a historical sequence, showing how the different approaches have evolved to meet the changing requirements of languages and applications.

In this topic, we use the term 'program' to refer to the source code that you might write in some programming language (including SQL) as well as its compiled, object code, which forms the basis of the executable application process that interacts with a DBMS. We will make the form of program explicit if it is not clear from its context.

The examples we provide relate to the University database and use very simple SQL statements, because the main focus of this topic is on how any kind of SQL, simple or complex, can be used in a programming context. It makes no difference to an explanation on how to retrieve data from a single column whether a table has two columns or two hundred columns. You will be assessed on the principles of what is involved rather than their application to complex problems.

2

Embedded SQL

So far in this course we have considered SQL mainly as an interactive language, in which an individual SQL statement is input using a database tool and then transferred to a DBMS for immediate execution and return of results. Apart from SQL routines, we have not had to be concerned with how data is returned by a DBMS because a database tool has simply displayed results in an appropriate format, that is, as a table.

Such a simple view is inadequate when you include SQL within a program. This use of SQL is the form in which it was initially developed, and it is known as **embedded SQL**. You may be relieved to know that, in many ways, the SQL that is embedded in programs is very similar to the SQL you have used interactively, and should be interpreted in the same way. However, there are certain features of embedded SQL that are different from what you have used so far, and it is those features that are considered in this section. In particular, there is a need to understand the relationship between SQL and the programming language that is used to write the program, as well as the means of transferring data between a compiled program and a DBMS.

2.1 Programming languages using embedded SQL

Before examining the details of embedded SQL, we need to consider some general issues relating to the programming languages that can be used. In particular, an essential requirement in the development of SQL was to be able to use the same SQL statements embedded within different programming languages — that is, as far as possible SQL should be independent of the language in which it was embedded.

From SQL:1992, reference to the standard programming languages Ada, C, COBOL, Fortran, MUMPS, Pascal and PL/1 has been explicitly included, though that does not mean SQL cannot be used with other languages. There are significant differences in the syntax of each language, as well as in the data types and data structures that each of them supports.

If we want the same SQL for different languages, that means there is not a C version of SQL and a separate Fortran version of SQL, and so on. Thus a program with embedded SQL is a hybrid, which is a combination of SQL and some other programming language – referred to as the **host language**. There needs to be a way of distinguishing SQL statements from those of the host language, which for most languages is done simply by prefacing each SQL statement by `EXEC SQL` and ending it with a semicolon.

In addition, because of the differences between programming languages, and particularly the limitations of the data structures in some languages, there is no possibility of transferring a complete table of data so that it can be processed by all of these languages. The only common feature of these languages that can be used for transferring data is a value of a simple data type. Thus a row of data has to be transferred as a number of separate values, one for each column in the table.

Even then, the data types differ between languages, so SQL has its own data types that are independent of any particular language. However, there is a defined mapping

between most SQL data types and those of each of the standard programming languages.

For example, the C data type `short` is equivalent to the SQL data type `SMALLINT`, `long` is equivalent to `INTEGER`, and a C character string, which is represented by a fixed-size array of characters, is equivalent to an SQL character string. For COBOL, data types are expressed in terms of `PICTURE` specifications of a numeric or character format, which have equivalent SQL numeric or character string data types.

This mapping between data types is required for a special kind of programming variable, called a **host variable**, that is declared as having a data type of the host language. A host variable is different from other variables in that it can be used in both host language statements and SQL statements. Host variables provide the means of transferring values between a DBMS – such as columns of a table, which use SQL data types – and the host language with its own data types. For example, host variables that could be used for the values from a row of the `course` table in the University database may be declared in a C program as follows:

```
EXEC SQL BEGIN DECLARE SECTION;
    char code[ 2] ;
    char title[ 20] ;
    short credit;
EXEC SQL END DECLARE SECTION;
```

The appearance of embedded SQL is not shown in the style of SQL statements because it is embedded as part of a program, and we use a different style to show that.

In this example, you should observe that the data types of the host variables must be mappable to those of the corresponding columns of the table, but there is no requirement for host variable names to be related to the corresponding column names, though it is advisable to make them similar since they are referring to data representing the same meaning.

Note that not all SQL data types have equivalent programming language data types. For example, most programming languages do not support the `DATE` data type, so a value of this SQL data type has to be cast as a character string for it to be transferable to a host variable. Also, programming languages do not support null (C has a null pointer, which is different) and this requires special treatment by using an indicator variable, which involves additional programming that we are not considering.

2.2 Processing embedded SQL

There is a further issue to address before considering how you might write a program with embedded SQL – that is, how the source code is processed to give an executable program. There are two main factors relating to this issue.

One factor in this processing is that an embedded SQL program is a hybrid, but the compilers that are used to produce an executable program cannot cope with such a mixture of languages – they are designed for, say, pure C or pure Fortran. There needs to be some mechanism to convert SQL statements into a form acceptable to a language compiler.

The second factor arises from a further requirement of SQL – it needs to be portable so that each SQL statement should work in the same way for different DBMSs. However, each DBMS is produced with its own interface designed by the vendor to implement the range of capabilities required to manage a database, with different ways in which these capabilities are invoked. This is called the **native interface** to the DBMS. Because the native interface is different for each DBMS, there is no standard way of converting SQL into a form that is understandable to a compiler.

The solution to this problem is that each DBMS vendor provides what is called a **pre-compiler** for each language. A pre-compiler processes a hybrid program of SQL and the host language source code into pure source code for that language by replacing all SQL statements with requests to the native interface supported by that DBMS. As a simplistic example, a pre-compiler might replace the embedded SQL statement

```
EXEC SQL DELETE FROM example_table;
```

with the following call to a routine that is provided by the native interface, which is supported by the vendor's DBMS:

```
CALL vendor_delete(example_table)
```

A compiler for the host language can now process the result of the pre-compilation without having to be aware of any SQL – it is pure source code. The compiled object code may then be executed as an application process that requests the DBMS to access the database according to the embedded SQL statements in the original source program.

Figure 2.1 shows the essential stages in the process of a program using the host language C to access a database managed by a DBMS that is supplied by a company 'vendor'.

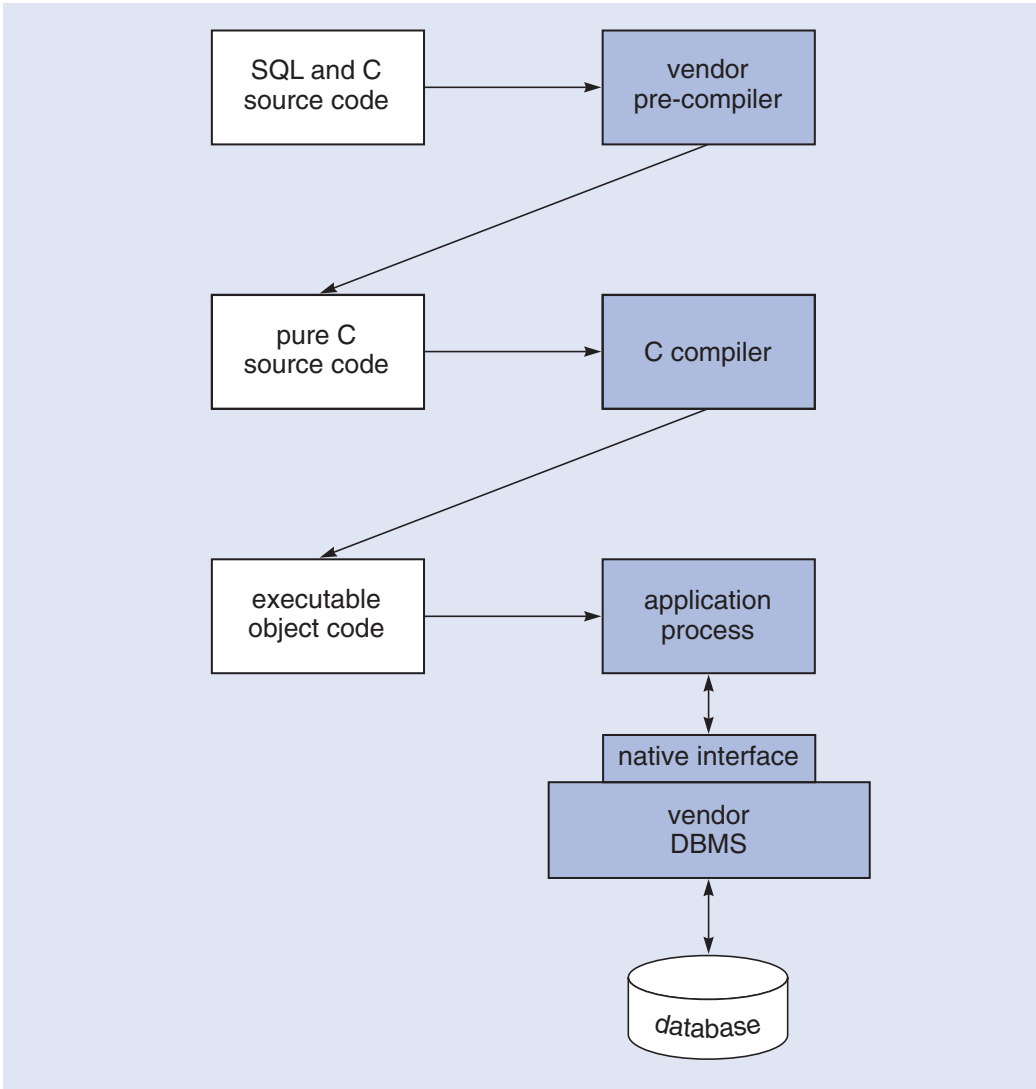


Figure 2.1 Processing embedded SQL

You should appreciate that Figure 2.1 is a simplified view of the process, and that more complex variations are possible. For example, it is implicit that a program is written for a particular database, so that a pre-compiler could check the embedded SQL statements against the database schema to ensure that the tables and columns referenced in the SQL exist in the database, in order to prevent errors during execution of the application process. In addition, a pre-compiler may process the embedded SQL to produce an executable plan for each statement that can be invoked every time it is executed, resulting in more efficient operation.



EXERCISE 2.1

It is a requirement of SQL that it is portable. Explain why it is only the source code for an embedded SQL program that is portable. Describe what must be done when an embedded SQL program, working with one vendor's DBMS, is required to work with another vendor's DBMS.

2.3 Embedded SQL statements

Having considered some of the general issues relating to the use of embedded SQL, let us now examine individual statements in more detail. There are two main differences between interactive SQL and embedded SQL that need to be taken into account – using host variables and manipulating data in terms of individual values, one row at a time.

We must initially note that the limitation on a program's manipulation of data does not prevent an embedded SQL statement requesting a DBMS to process complete tables of data. For example, a single embedded **UPDATE** statement may request that all values of the column **mark** in the **examination** table are increased by 10; this is valid as an embedded SQL statement because a DBMS can perform this update for every row in the table without any further interaction with the requesting program.



EXERCISE 2.2

There are two forms of **INSERT** statement that add data to a table, which are either (a) an explicit collection of values for a row, or (b) data selected from one or more tables according to a specified query. For each of these two forms of the **INSERT** statement, explain whether it is valid as an embedded SQL statement.

Thus, as we have already indicated, many of the SQL statements with which you are familiar can be used in the same way as embedded SQL, but what we can now consider is how host variables are used with these statements.

Using host variables

In previous use of SQL, you formulated statements that included only literal values — that is, constant values such as c1 for a course code, or 2 for a region number. Such literals may occur in a program, but more generally a program that may be executed many times is written using variables that can be assigned different values each time they are used. That is the role of host variables, which are used in embedded SQL statements for interaction with a DBMS as well as in host-language statements for all other kinds of processing, such as input, output and calculation.

When a host variable is used in a host-language statement, its name is that given in an SQL `DECLARE SECTION`, but when used within an SQL statement the name is prefixed by a colon to distinguish it from SQL names. For example, the host variable named `code` would appear in a host-language statement in that form, but in an embedded SQL statement it would appear as `:code`.

As an example, let us consider what would be required for a program to add a new course to the University database. Initially, such a program would need to connect to the database, and then it would request user input for the values required for the columns of the new course, read that input (perhaps validating it) and assign the values to the host variables that have been declared with appropriate data types, such as those shown in the example SQL `DECLARE SECTION` for a C program. The host variables can then be used in an **INSERT** statement, as follows:

```
EXEC SQL
INSERT INTO course (course_code, title, credit)
VALUES (:code, :title, :credit);
```

In this statement, the values of the host variables, having been assigned within the program, are passed to the DBMS for it to use when adding a new row to the **course** table. In order to be able to do so, it is necessary that the data type of each host variable is compatible with the data type of the column for which it is providing a value. Note that this correspondence between host variable and column is according to position, not name, so there is no need for any matching between the names of the host variables and the names of the columns. There can be no confusion if the names are actually the same because a colon is required for a host variable name.

EXERCISE 2.3

What else must be included in a program when you use `:var` in an embedded SQL statement within the program?



You can see another way of using a host variable in embedded SQL in the following **DELETE** statement, where the value that has been assigned to the host variable `code` determines the row of the **course** table that is deleted.

```
EXEC SQL
DELETE FROM course
WHERE course_code = :code;
```

EXERCISE 2.4

Give an embedded SQL **UPDATE** statement for the **course** table so that the course whose title is given by the host variable `title` has a credit value given by the host variable `points`.



The use of host variables for embedded **SELECT** statements is different from these update statements because data is transferred in the reverse direction – the DBMS is providing values to be assigned to the host variables. Let us first consider the situation where a query is known to result in, at most, a single row, such as when the primary key of a table is used for selection. In that case, the value for each column of the result can be assigned to a suitably declared host variable by an **INTO** clause, which is the same as that used for SQL routines in Section 7 of *Block 3* except for the inclusion of host variables, as you can see in the following example.

```
EXEC SQL
SELECT title, credit
INTO :title, :credit
FROM course
WHERE course_code = :select_code;
```

In such an embedded **SELECT** statement, it is necessary for the host variables given in the **INTO** clause to correspond in position and data type (not name) to the columns given in the **SELECT** clause. Execution of this example query would result in the DBMS assigning the values retrieved from the specified row of the **course** table to the corresponding host variables, which can then be used within the rest of the program for calculation or display by the host language.

In this example, you should appreciate that the one embedded SQL statement may be executed many times within a program by placing it in a loop structure that is controlled by host language statements. Each execution of the loop could, say, request a value for the host variable, `select_code`, from the user of the program and then retrieve the specified row. The overall effect is that many rows of the **course** table are retrieved one at a time. However, this form of an embedded query statement cannot be used when many rows are retrieved at once. In this case, a more complex form of retrieval is required.

Using cursors

Section 7 of *Block 3* describes how cursors are used in the context of SQL routines. For embedded SQL, the concepts are the same but the details differ in two respects. First, in the same way as a single-row query, host variables are used instead of SQL variables in an **INTO** clause. Secondly, you cannot use SQL control statements to manage the execution sequence; you have to do that using the host language.

For example, if we wanted a program to display the course codes and titles of all the courses in the **course** table, each having a credit of 60 points, in title order, we would start off by declaring a cursor for this purpose, as follows:

```
EXEC SQL
DECLARE long_course CURSOR FOR
  SELECT course_code, title
  FROM course
  WHERE credit = 60
  ORDER BY title;
```

Before being able to access any of this data, an **OPEN** statement is required to initiate retrieval of data from the database:

```
EXEC SQL
OPEN long_course;
```

Then a **FETCH** statement needs to be executed repeatedly so that each row of the query result has its column values assigned to host variables:

```
EXEC SQL
FETCH long_course
INTO :code, :title;
```

When all the rows have been fetched, the cursor should be closed:

```
EXEC SQL
CLOSE long_course;
```

Thus the embedded SQL statements are essentially no different from those you could write within an SQL routine, but what we have not shown are the host language

statements you would need to include in the program to control the execution sequence, which obviously depend on the language you are using. The most important aspect of this control is a looping mechanism that repeats the **FETCH** statement, perhaps with other host language statements to display the results in the host variables, until all rows have been retrieved.

EXERCISE 2.5

By reference to the use of cursors in Section 7 of *Block 3*, explain how a program would determine when all rows have been retrieved.



Having explained that the processing of a query using a cursor requires control statements in the host language, we need to emphasise that it is the only way in embedded SQL. Section 7 of *Block 3* describes how SQL routines support the **FOR** statement, which combines a loop mechanism and a cursor declaration without the need for the **OPEN** and **FETCH** statements, but that is not allowed in embedded SQL.

The mention of `SQLSTATE` in the solution to Exercise 2.5 introduces another aspect of using embedded SQL, which we are not going to consider any further; but you need to be aware of some of the complexities involved in developing database applications. As well as the value `02000` signalling that there is no data, `SQLSTATE` can have a range of values indicating all sorts of exceptions that might occur when a DBMS is accessing a database. A database tool executing interactive SQL simply tells its user of any exception, assuming that such a user can understand the problem and respond to it. However, a program should be written in a way that does not make such assumptions. Thus an embedded SQL program should test `SQLSTATE` for all kinds of possible exceptions and include code to handle them, if only to explain the situation to the user of the program.

2.4 ODBC (Open Database Connectivity)

Before we start, we need to emphasise that **ODBC** is not embedded SQL, though it is used embedded in programs and it does involve SQL. In this brief overview of ODBC, we focus on its use of SQL and how it differs from embedded SQL. It is included in this section as a link between embedded SQL, as we have just described, and the next section where we examine JDBC, which was developed for the programming language Java in a similar way to the use of ODBC for other languages.

So what is ODBC? In general computing terms it is an **application programming interface** (API). The initials stand for Open Database Connectivity, and it is much more widely known by these initials than the term used for the SQL standard version, which is SQL/CLI (Call-Level Interface). ODBC was developed by DBMS vendors, mainly Microsoft, to respond to the problem with embedded SQL, which was identified in the solution to Exercise 2.1 – a compiled embedded SQL program is not portable between different vendors' DBMSs. Software developers wanted to produce compiled 'shrink-wrapped' packages, such as for payroll or stock control, that would work with any DBMS. This is not possible with embedded SQL.

The ODBC solution to this problem is to specify a standard interface through which a program can request access to a database. The source code for the program is written with requests to the ODBC interface expressed using normal programming language invocation so that it can be compiled without any pre-processing. The compiled object code, executing as an application process, submits requests for database access via the ODBC interface.

You can think of an ODBC driver as comparable to a printer driver, as a component that enables a connection between an application program and some 'device', such as a printer or a database.

However, access to a database is still via a DBMS, which has its own native interface, therefore the ODBC interface is provided by a component called an **ODBC driver** that is placed between an application process and a DBMS. There are different kinds of ODBC driver, but their main purpose is to accept requests from an application process expressed in terms of the standard ODBC interface, convert the requests into a form that can be processed by the DBMS native interface, and then return the relevant results. Figure 2.2 shows the essential components when ODBC is used with the language C.

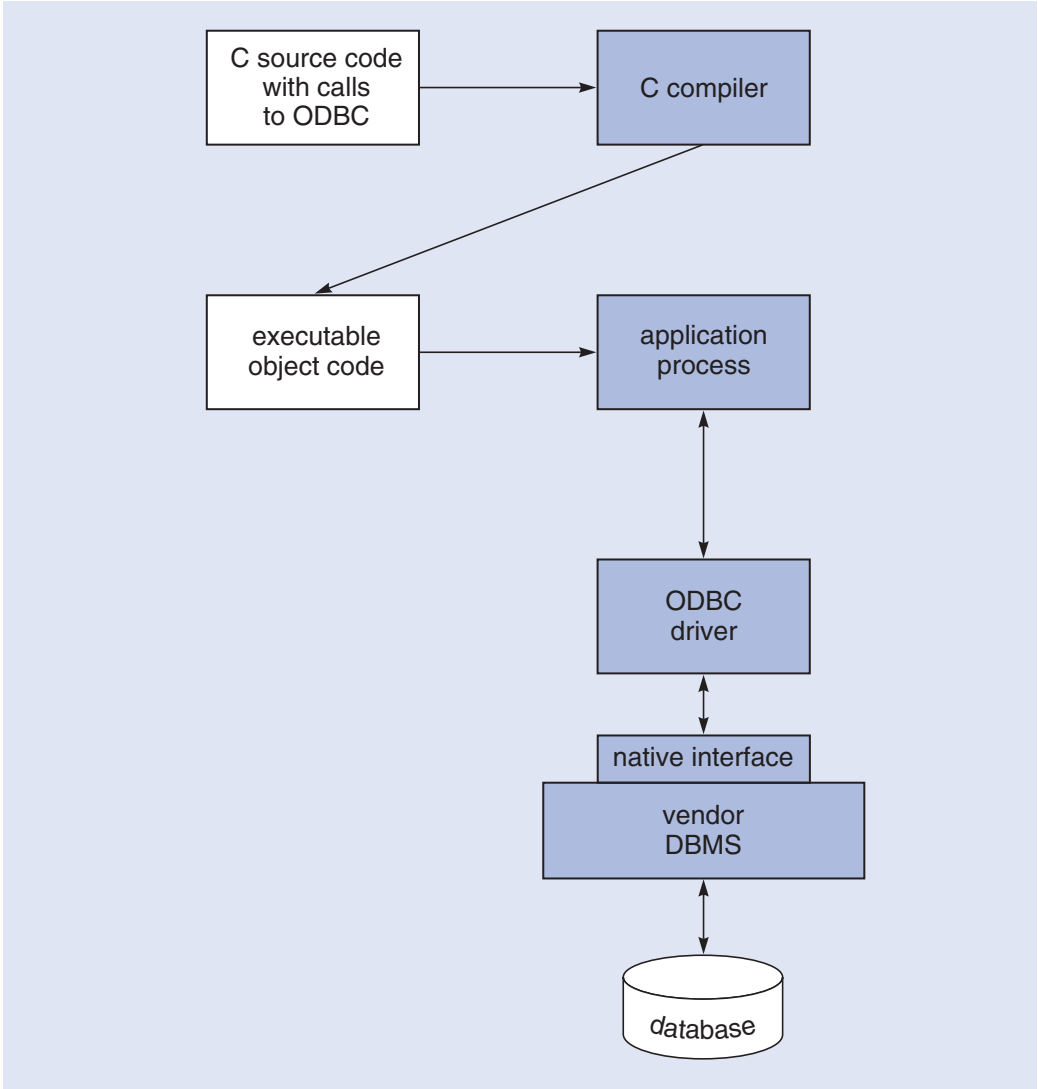


Figure 2.2 Database access with ODBC

You need to appreciate that Figure 2.2 includes only those components that participate directly in accessing data in a database. Another component that is required as part of ODBC processing is an ODBC manager, with which an application process first interacts to establish a connection to a database. This connection is to an **ODBC data source**, which is a previously declared combination of an ODBC driver appropriate for the DBMS to be used and the database to be accessed. The specification of a data source may also include details, for example, a user identifier and password, and other management information such as whether to start and stop the database automatically.

EXERCISE 2.6

Explain why an ODBC driver for a data source needs to be 'appropriate for the DBMS to be used'.



When you compare Figure 2.2 with Figure 2.1, the significant difference is that there is no pre-compiler, which is the vendor-dependent part of using embedded SQL that is not required when using ODBC. Of course, you start off with a program written in some programming language (ODBC is based on using the language C, though it can be used with other languages that can call C routines) but the source code is a pure language that can be compiled directly without requiring any pre-compilation.

So where is the SQL that determines what database access is required? ODBC enables SQL to be specified by routines that accept an SQL statement as a character string argument. One of these routines invokes the immediate execution of an SQL statement, such as:

```
SQLExecDirect(my_handle,
              "SELECT course_code, title FROM course", 40)
```

In this example, the SQL statement is obviously the second argument, which we have given as an explicit character string for clarity but it could also be a character string variable to which an SQL statement has been assigned. The third argument is just the length of the statement character string, and the first argument is a variable that has been previously declared (actually a resource that has to be allocated) as a statement 'handle', which is a means of referencing the statement. Assuming that a connection to a data source has been established, execution of this routine causes the SQL statement to be passed to the connected ODBC driver and hence to the DBMS for it to respond to the request. ODBC provides access to the result of the query, which ODBC calls a **result set**, by using a cursor. However, an ODBC cursor is used implicitly for processing a result set; there is no need for an explicit declaration as in embedded SQL. An ODBC application program has to fetch the rows of the result set in a similar way to embedded SQL, but it does so by referencing the statement handle rather than the cursor, as in:

```
SQLFetch(my_handle)
```

Notice that this fetch routine moves the cursor from row to row in the same way as **FETCH** in embedded SQL, but it does not specify where the data in the fetched row should go. One way of accessing the data that has been fetched is by using a routine that obtains an individual value from a row according to its position. This routine, `SQLGetData`, requires the statement handle, the column number, the data type of the value and a variable (a buffer, which is a memory location to hold it) to which the value is assigned. For example, getting the data from the first column of a fetched row for statement `my_handle` requires a routine request like:

```
SQLGetData(my_handle, 1, ... ..)
```

Another way in which data from a fetched row of a result set is made available to a program is by 'binding' a column (again, specified by position) to a variable (also a buffer) before executing the fetch routine, in which case `SQLFetch` places the value for that column in the specified buffer. More complex options include declaring a structure for the whole row and using arrays to enable multiple rows to be fetched – a **rowset**.

An SQL update statement can also be executed using `SQLExecDirect`, but the details are not relevant for this overview. What is relevant is to notice that, whatever the SQL statement might be, it is passed to a DBMS as a character string so that it has to be processed by the DBMS to give a plan before it can be executed. This can be inefficient if a statement is executed many times, so ODBC provides an alternative way

of executing SQL statements that allows a statement to be prepared for execution, as in this example based on the same query as above:

```
SQLPrepare(my_handle,  
           "SELECT course_code, title FROM course", 40)
```

Having prepared an SQL statement in this way, it can then be executed as many times as required simply by referencing the statement handle, as in:

```
SQLExecute(my_handle)
```

After each execution of a prepared statement, each resulting result set can be processed in the same way as we have described for the immediate execution of a statement. However, this is not a good example to demonstrate the need for repeated execution because it gives the same result set each time. An extension to this example query, such as the addition of a **WHERE** clause with a parameter, would enable different results to be produced for each value of the parameter, but such a parameter is implanted in the character string of the statement rather than being a host variable as for embedded SQL. We are not considering exactly how this is done, but we return to this issue when examining a similar situation for JDBC.

Processing SQL statements as character string arguments for ODBC routines leads to another significant difference between embedded SQL and ODBC. An embedded SQL statement is part of a compiled program and no part of it, such as a table name or a column name, can be changed during execution of the program. It is known as **static SQL**. ODBC can be used in the same way, written with explicit SQL statements that are not changed, but it is also possible to write a program that constructs a character string in some way, perhaps involving a dialogue with a user, which can then be executed as an SQL statement. This is known as **dynamic SQL**, and it introduces the need for much more complex programming because the programmer does not know what such a statement might be requesting a DBMS to do, and so has to use a range of ODBC routines to find details such as the number of columns returned by a query, the data type of each column, and so on. (Actually, there are embedded SQL routines that support the use of dynamic SQL, but ODBC is normally used in such a situation.)



EXERCISE 2.7

Briefly describe the differences in the way embedded SQL and ODBC enable data to be transferred from a DBMS to a program, and in each case explain how the columns in the tables in a database and the variables in a program are associated.

2.5 Summary

To summarise this section, embedded SQL statements differ from the interactive SQL you have used so far in just a few ways. First, it is always necessary to consider the host variables that are required and provide a declaration of the data type for each one of them. Then, apart from queries that result in many rows, embedded SQL statements are closely comparable to the use of SQL interactively. However, cursors are required to handle queries that result in the retrieval of many rows, which need appropriate control mechanisms for repeated processing of each row. The overall effect is that in many situations you can continue to think of embedded SQL as a relational (or table) language, but at times it is necessary to think of processing the individual values from a row of a table.

ODBC routines may look very different from embedded SQL, but they support the use of static SQL statements in a way that is similar in many respects to embedded SQL. There is a significant difference in the need for explicit connection to a data source and the way that data is transferred between a DBMS and a program. In addition, ODBC supports the use of dynamic SQL. ODBC also provides control over the processing of SQL that is not possible for embedded SQL, so there is a choice to be made for each statement as to whether it is to be executed directly or prepared first.

3

Java and databases

In this section, we examine a number of ways in which the programming language Java can be used to access data in a database. Java is treated differently from the other programming languages that use embedded SQL or ODBC because it is an object-oriented language. Java programs are constructed in terms of classes, which have variables and methods that can be executed; there are also interfaces, but they have to be implemented by classes and therefore we ignore the distinction in this description. Java is strongly typed, so all variables have a type, which may be either a primitive type, such as `int` (for 32-bit integers), or a reference type, such as the class `String` (for character strings). Java programs are compiled so that they can be executed by a **Java Virtual Machine** (JVM), which can be implemented in any machine environment and provides portability for compiled Java programs.

Most of this section relates to the JDBC application programming interface. The initials stand for Java Database Connectivity, and its name indicates its close relationship to ODBC. JDBC was initially developed by Sun Microsystems, the originator of Java, and is now the responsibility of the Java Community Process (JCP). Note that the JCP is distinct from the International Standards committee that is responsible for SQL.

After considering JDBC, we briefly describe the complementary SQLJ and some other ways in which Java can be used for database access.

3.1 JDBC

As we consider the details of JDBC, you should remember the caveat given in Section 1: we are not aiming to produce complete programs, such as might involve calculations, fancy displays or handling exceptions. Thus we are showing you examples of the Java statements required for database access with the aim of demonstrating the steps involved and explaining their purpose; you are not expected to understand the detailed syntax.

Before a database can be accessed, some initial work is required.

Getting started

Any Java class needs to declare that it requires methods to be found in the Java class libraries. When using JDBC this is specified by:

```
import java.sql.*;
```

Once this declaration is included, a program with JDBC statements is just an ordinary Java program that can be compiled and executed in the usual way. Furthermore, like an ODBC program, it is portable so it can be used in different environments.

In addition to the JDBC classes, a program requires a **JDBC driver**, which plays a similar role to an ODBC driver as shown by the simplified model in Figure 2.2. Thus a JDBC program must include an initial step of activating an appropriate driver for the DBMS to be used. One way in which this is done is by a statement such as:

```
Class.forName("sun.jdbc.odbc.JdbcOdbcDriver");
```

The effect of such a statement is that the method `forName` (of the class named `Class`) searches for a class having a name given by the argument and then loads an instance

of that class. In this example, the name refers to a driver that is available as part of Sun's Java development kit. You may be curious why the name includes ODBC; this is because the driver we have given as an example is known as a **JDBC-ODBC bridge**, which means that it provides support for JDBC by invoking ODBC routines. While not the most efficient kind of driver, it has the benefit of being generally available and usable with any DBMS that has an ODBC driver.

You need to appreciate that there are different kinds of JDBC drivers available and other ways in which they can be loaded. In particular, a more recent way of referencing a driver is by a `DataSource` interface, which is now the preferred way of using a driver, but it involves the Java Naming and Directory Interface (JNDI) and we do not want the distraction of explaining what that involves. The important point for you to remember is that a JDBC driver is required before trying to access a database with JDBC.

The next step is to establish a connection to the database to be accessed, which is represented as an instance (i.e. an object) of a JDBC class called `Connection`. A class called `DriverManager` keeps track of drivers that have been loaded, and it has a method named `getConnection` that can be used to connect to the University database and give a `Connection` object, which we can assign to the variable `univConnection` as follows:

```
univConnection = DriverManager.getConnection(
    "jdbc:odbc:UniversityDSN", "m359", "m359");
```

This statement creates a connection to the database identified by the first argument, which is a URL that must start with `jdbc` and then continue in a way that depends on the driver being used: in this case, since it is a JDBC-ODBC driver, the database is identified by its ODBC data source name – `UniversityDSN`. The second and third arguments are the user identifier and password respectively, which are required to connect to the database. If the user identifier and password are included in the ODBC data source, then you can use another version of `getConnection` that just needs the URL with the data source name.

Once a connection is established, a program can then access a database by executing SQL statements. In the following examination of different kinds of SQL execution, we assume that the `Connection` object, which is held as `univConnection`, has been established as just described.

Immediate execution

Similar to ODBC, execution of SQL using JDBC requires a `Statement` object, which is an instance of the class `Statement` that is associated with a connection to the database being accessed. The `Connection` class enables you to create a `Statement` object with its `createStatement` method, which can be used with the example connection, `univConnection`, to create a `Statement` object and assign it to the variable `st`, which must have been previously declared as type `Statement`, as follows:

```
st = univConnection.createStatement();
```

Having created this `Statement` object, it can be used to execute an SQL query by using the `executeQuery` method of the `Statement` class, as in the following example:

```
results = st.executeQuery(
    "SELECT course_code, title, credit "
    + "FROM course");
```

In this example, the SQL query given as the argument is sent to the connected DBMS, then it is executed and the result is returned to the variable `results`, which must have been declared as type `ResultSet`. The `ResultSet` class provides an explicit representation of the result set concept in ODBC and is processed in a similar way

The variable `univConnection` must have been previously declared as type `Connection`.

You need to be careful to distinguish the general use of 'statement' to refer to a step within a Java program and its specific use to refer to an instance of the `Statement` class, which we usually write as a `Statement` object.

The `+` in this statement concatenates the two strings.

Because there is no ambiguity, we use 'result set' to refer to an instance of the `ResultSet` class, but you should remember that it is really a `ResultSet` object.

JDBC has no concept of domain, so it is the SQL base data type of a column that determines the Java type to which it can be assigned.

using an implicit cursor that references individual rows, as described in Section 7 of *Block 3*.

The `ResultSet` class has a number of methods to support processing of a result set. The most important one is the method named `next`, which moves the cursor to the next row of a result set, and it becomes the **current row** (remember a cursor starts before the first row). The method `next` gives a result that is the Boolean value `true` if there is another row, or the value `false` if there is no other row (meaning that the cursor has reached the end of the result set). This method can be used in a simple loop, such as:

```
while (results.next())
{
    ... ..
    ... ..
}
```

The effect of this kind of loop is to allow each row of a result set to be processed by the Java statements within the block `{ ... }`. For that processing, we turn to the methods of `ResultSet` that enable a value to be obtained from an individual column within the current row. While this concept is similar to the way ODBC works, one significant difference is that JDBC provides separate methods for each data type, named as 'get' followed by the type.

For example, the column `course_code` is a character string. So a value from this column in the current row can be assigned to the variable `courseCode`, which must have been declared as type `String`, as follows:

```
courseCode = results.getString(1);
```

This statement references the required column by position (it is the first column in the `SELECT` clause), which is the same as used by ODBC, but JDBC also provides an alternative that allows the column name to be used, as in:

```
courseCode = results.getString("course_code");
```

We will follow the practice of always using a column name, making the association with the query explicit and more understandable, but you need to appreciate that there are circumstances in which a programmer may not know the names of the columns in a query and may have to use column positions (though note that there are ways of obtaining column names).

We can demonstrate another method for obtaining values from a result set by considering the column `credit`, which has a data type of `SMALLINT`. A value of type `SMALLINT` maps directly to the Java type `short` (a 16-bit integer), so you can assign the value of the column `credit` in the current row to the variable `credit` declared as type `short` using the following statement:

```
credit = results.getShort("credit");
```

Notice that there is no conflict between the name `credit` for the variable and the same name being used to refer to the column, because the column name is just a character string in this statement.

In this example, we have used the direct mapping from the column value to a variable – that is, they have equivalent representations – but other mappings are possible. Thus a column value of type `SMALLINT` can be assigned to a Java variable of type `int` by using the method `getInt`, but the converse of assigning a column value of type `INT` to a variable of type `short` is not possible because such a variable is not capable of accepting all possible values from the column. An alternative mapping for the column `credit` in our example is to assign it to a `String` variable using the method `getString`, which automatically transforms the `SMALLINT` value to a character string

representation. This option of transforming to a Java character string is generally available for most SQL data types.

To show how these statements can be put together to process the rows of a result set, including the column **title** assigned to the `String` variable `courseTitle`, the following code displays all the rows of the table using the system output:

```
while (results.next())
{
    courseCode = results.getString("course_code");
    courseTitle = results.getString("title");
    credit = results.getShort("credit");
    System.out.println(courseCode + " " + courseTitle
                      + " " + String.valueOf(credit));
}
```

Notice that the printed string is a concatenation of the columns (with spaces) and, having chosen to assign the column **credit** to a variable of type `short`, we use the expression `String.valueOf(credit)` to convert the value to a string.

There are many other methods for the class `ResultSet`, such as `close` – as you might expect from the need to close a cursor. One possibly misleading method to note is `getRow`, which does not get a row at all but returns a number that is the position of the current row in the result set – 1 for the first row, and so on – which is called the **row number**.

EXERCISE 3.1

For embedded SQL, a cursor has to be opened before the results of a query can be accessed. Describe how JDBC provides the equivalent of opening a cursor.



EXERCISE 3.2

Explain how the row order that gives the row number for a result set is determined

EXERCISE 3.3

Suppose you wanted to list the details of courses having a credit that is less than 40. Describe (no code required) simple changes to the example statements given in this section so that this condition is applied first by the DBMS and secondly by the Java program.

The two alternative ways of applying a condition, as described in the solution to Exercise 3.3, demonstrate one significant issue that you need to be aware of when assessing such choices. For the DBMS solution, the program has to process only those rows satisfying the condition, whereas for the alternative in which the program applies the condition, it has to process all the rows in the database table. While this is a very simple example, for a much larger table with many rows and a condition that results in just a small proportion of those rows, you should recognise that it is more efficient for the DBMS to apply the condition, particularly if it has indexes defined for the columns involved.

Thus a useful general principle is to write queries such that a DBMS is required to do as much of the processing as possible.

To see the impact of this principle, let us now consider how you might approach the task in Exercise 3.3 in a more generally applicable way, so that instead of requesting a list of courses with credit less than 40, we request a list of courses with credit less than some variable, say `limit`, declared as the Java type `int`. We are not interested in how

this variable may be assigned a value, but possibly it would result from some dialogue with a user. However, the variable cannot be included in the query in the way a host variable can be included in an embedded SQL statement, because the query is just a character string in the `executeQuery` method and we cannot place a variable within a character string.

There are two approaches to resolve this problem. One approach is to construct the character string for the query from its component parts, so we can build the text of the required query and assign it to a variable that we have chosen to name as `sql` (declared as type `String`) as follows:

```
sql = "SELECT course_code, title, credit "  
      + "FROM course "  
      + "WHERE credit < " + String.valueOf(limit);
```

Notice that the value of the variable `limit` is converted to a string in order to concatenate it with the rest of the query.



EXERCISE 3.4

If the variable `limit` has a value 10, what is the value of the variable `sql`?

Having constructed the query as a string, it can then be executed using the same method as before, with the variable `sql` providing the text of the query:

```
results = st.executeQuery(sql);
```

The results of executing the query can also be processed as before.

This construction of the text of a query provides a very simple example of what we referred to as dynamic SQL when describing ODBC. There is no restriction on how such a string may be constructed as long as it contains valid SQL so it can be executed by a DBMS.



EXERCISE 3.5

Compare and contrast the safety of an embedded SQL program with a dynamic SQL program (either ODBC or JDBC) in terms of only being able to perform an approved task.

For the second approach to the problem of including a variable in a query, we need to first start by introducing prepared statements.

Prepared statements

The reason for having prepared statements in JDBC is the same as for ODBC – it is more efficient in some situations, enabling an SQL statement to be checked and an execution plan prepared so that it may then be executed many times without having to repeat this preparation. Let us consider how to prepare our example of a simple query. It involves the class `PreparedStatement`, a subclass of `Statement`, which means that an instance behaves as a `Statement` object in most respects but has the additional feature of being created with a query that is prepared for execution. For the example database connection, a `PreparedStatement` object can be assigned to a variable `prepSt` (declared as type `PreparedStatement`) as follows:

```
prepSt = univConnection.prepareStatement(  
    "SELECT course_code, title, credit FROM course");
```

Once prepared, the query can then be executed as many times as required using the same `executeQuery` method as before, except that now the query has been incorporated into the `PreparedStatement` object, held as `prepSt`, so all you need is:

```
results = prepSt.executeQuery();
```

The result set produced by executing a prepared statement is no different from that produced by the immediate execution of a query, and therefore can be processed in the same way as previously described.

However, what is different for a prepared statement is the way that a variable can be included in an SQL query. Since a prepared statement only uses the text of a query at the time of its preparation, any variable included in the string (as in the example using `limit`) would have its value at that time used in the preparation, but any subsequent change to such a variable would have no effect on the prepared statement.

In order to allow a variable to determine the results of each execution of a prepared statement, a **parameter marker** is used. A parameter marker is represented in the text of a query by a `?`, as shown in the following form of the example query, in which we want to use the variable `limit` to apply a condition on the course credit:

```
sql = "SELECT course_code, title, credit "  
      + "FROM course "  
      + "WHERE credit < ?";
```

The variable `sql` is a character string (with its embedded `?`) and can be used, as in the previous example, to prepare the statement as follows:

```
prepSt = univConnection.prepareStatement(sql);
```

In this case, the prepared statement cannot be executed because there is no value for the parameter marker – that is, the `?` in the query – and one needs to be assigned. The `PreparedStatement` class has a number of methods that are used for this purpose. Each of these methods is named as ‘set’ followed by the type of the value to be assigned to the parameter marker. For example, the following method, applied to the prepared statement `prepSt`, assigns the value of the variable `limit` to the parameter marker:

```
prepSt.setInt(1, limit);
```

Notice that the method is providing a value for the DBMS to use in executing the query, and so you need to think of how the parameter marker is used in the query, not about embedding it in the text of the variable `sql`. In this example its value is being compared to an SQL **SMALLINT** value and so the `setInt` method is appropriate and assigns a value of `int` data type to the parameter marker in the query. As described for the get methods of `ResultSet`, JDBC can transform between comparable data types, but the general transformation of a value of type `String`, which is possible for gets, is not acceptable for sets (except when a string value is required).

The first argument of a set method is an integer that specifies the position of the parameter marker in the query, and in this example there is only one parameter marker so it has to be 1, though in general there can be many, numbered sequentially. The second argument is the value to be assigned to the parameter marker, which in this example is given by the variable `limit`. Once the parameter marker is set, the prepared statement can be executed as before using the `executeQuery` method.



EXERCISE 3.6

Write the sequence of statements required to prepare and then execute a query that will display the title and credit for a course whose course code is given by the `String` variable named `code` (whose value you can imagine has been assigned by a user dialogue).

You should appreciate that the solution to Exercise 3.6 does not constitute a complete program – just the essential elements of it. Also note that we have used our knowledge of `course_code` being the primary key of `course` to simplify the solution, since the result set has only one row and there is no need to have a loop to process all the rows. More generally, you would expect that a program would be written to repeat execution of the query many times, which is the main justification of using prepared statements.

We can also use this example to introduce one further feature of result sets. So far, we have only used the `next` method to move a cursor forward through the rows of a result set, but there is another kind of result set that has a cursor that also can move backwards or directly to any row location, such as the first, last or seventh row. It is called a **scrollable cursor**.

This kind of result set requires that certain properties are specified when creating the `Statement` object being used to generate the result set, which is as follows:

```
prepSt = univConnection.prepareStatement(sql,
    ResultSet.TYPE_SCROLL_INSENSITIVE,
    ResultSet.CONCUR_READ_ONLY);
```

The `createStatement` method has directly comparable properties.

This form of the `prepareStatement` method has an additional two parameters that are actually just integer codes, but the `ResultSet` class defines these two constants for this purpose so you do not have to remember what values are required.

The first additional parameter determines the type of result set that can be generated from this statement, and at this point our focus is on the scrolling capability specified by `TYPE_SCROLL_INSENSITIVE`, which is precisely what we have just described – the ability of a scrollable cursor to move in various ways around a result set. Thus, with the `PreparedStatement` object `prepSt` as just defined in the solution to Exercise 3.6, we could have obtained the required (first) row by using the following method:

```
results.first();
```

Note that the aspect of this parameter signified by `INSENSITIVE` relates to whether changes to the result set are accessible through the cursor but, in our example, changes are prevented by the second additional parameter – specifying that the result set is to be read-only and so cannot be updated – so sensitivity to change is not relevant when you are only considering queries. We shall return to this issue when we next consider updating.

Before concluding our consideration of JDBC queries, we must emphasise that we have only described the basic concepts and there are many more options to consider in developing JDBC programs. For example, there is a method `getMetaData` for result sets that gives a `ResultSetMetaData` object containing many details about a result set, such as the name and data type of each column.

To summarise, the sequence of steps involved in a JDBC query, after having acquired a database connection, is as follows:

- ▶ A query is formulated as text, which may be assigned to a `String` variable or used directly as a literal argument in subsequent steps.
- ▶ A statement object is created or prepared, with a choice of whether a scrollable cursor is required.

- ▶ If a prepared statement includes parameter markers, a set method for an appropriate data type is used to assign a value for each parameter marker.
- ▶ The statement (and hence its associated query) is executed, giving a result set.
- ▶ A result set is processed by moving its cursor from row to row.
- ▶ The value for a column in the current row is obtained by a get method with an appropriate type for that column, referenced by position or name.

Updating

In updating a database using JDBC, including inserting and deleting, there are two significantly different approaches and both are based on the concepts we have just described for queries.

First, we can consider the execution of an SQL update statement, which is directly comparable to the execution of a query except for one important difference – no result set is produced. Thus a different method, `executeUpdate`, is used. It can be used for any kind of update statement and produces a result that is the number of rows affected by the update.

As for queries, both immediate execution and prepared statements can be used for updating. For example, suppose we want to add another row to the `course` table. This can be done by creating a `Statement` object for the University database connection in the same way as for a query, which we can assign to the variable `st`, and declaring the variable `result` as `int` data type. Then immediate execution of the given SQL statement inserts the specified row into the `course` table:

```
result = st.executeUpdate(  
    "INSERT INTO course "  
    + "VALUES ('c9', 'Testing', 30)");
```

Note that the integer variable `result` is different from the result set variable `results`.

EXERCISE 3.7

What value should you expect the variable `result` to have on completion of this statement?



While this example uses a literal string for the SQL statement, any kind of update statement can be constructed by concatenating character strings, as we described above for queries. Alternatively, you can prepare an update statement. As an example of this, let us consider how to delete a course. You start with the required SQL statement, such as:

```
sql = "DELETE course WHERE course_code = ?";
```

Then a `PreparedStatement` object, which we can assign to the variable `prepSt`, is created in the same way as in preparing a query:

```
prepSt = univConnection.prepareStatement(sql);
```

The parameter marker must have a value before the statement can be executed, which is set in the same way as for a query, so if we want to specify that the new course we have just inserted is to be deleted, we can do so with:

```
prepSt.setString(1, "c9");
```

Note that, in general, a variable would be used in setting the value for the parameter marker so that the prepared statement can be used for multiple executions. Once set, the `executeUpdate` method can then be used for the prepared update statement to execute the delete statement as follows:

```
result = prepSt.executeUpdate();
```



EXERCISE 3.8

For a course code specified by the variable `code`, give the statements required to update the course with that course code so that it has a new title given by the variable `newTitle`.

The approach to updating which we have just described is based on executing some SQL statement that determines what is updated. You should note that the `executeUpdate` method also allows an SQL data definition statement, such as **CREATE TABLE**, to be executed. In this case, the result is 0.

Alternatively, there is a different approach to updating that involves the execution of a query to generate a result set, then using some JDBC methods to make a change to the result set and applying that change in the result set to the database.



EXERCISE 3.9

Bearing in mind the restrictions placed on the kinds of view that are updatable, what restrictions would you expect to be placed on a query that generates a result set to which this alternative approach could be applied?

Even if a query satisfies the restrictions given in the solution to Exercise 3.9, the result set that is produced by executing it is not automatically updatable – there has to be a prior declaration that an updatable result set is required. That declaration is done when creating a `Statement` object, as in the following example:

```
st = univConnection.createStatement(
    ResultSet.TYPE_FORWARD_ONLY,
    ResultSet.CONCUR_UPDATABLE)
```

This form of the `createStatement` method is similar to the example of preparing a statement for using a scrollable cursor, though in this case we have specified that the cursor only moves forward – that is, it is not scrollable. This time we are focusing on the second parameter, which specifies concurrency. However, this term is misleading because it implies that there are a number of simultaneous users, but it is required even when there is just one user because the important factor is whether a result set is to be updatable or not.

Having created this `Statement` object, it can be used to execute a query as previously described:

```
results = st.executeQuery(
    "SELECT course_code, title, credit "
    + "FROM course");
```

In this case, the result set `results` is updatable, and has some additional capabilities for updating. For example, if we want to insert a new row into the `course` table, we first add it to the result set by making use of a special **insert row**. This row has the same columns as the result set but initially they do not have any value. The insert row cannot

be reached by normal movements of a cursor, such as using the method `next`, but a cursor for an updatable result set can be moved there by the following method:

```
results.moveToInsertRow();
```

Once the cursor has been moved to the insert row, each of the columns in that row can be assigned a value by using an update method for an appropriate data type, as in the following example for each of the columns of the `course` table:

```
results.updateString("course_code", "c9");
results.updateString("title", "Testing");
results.updateShort("credit", 30);
```

These methods assign the value given as the second argument to the column named by the first argument. However, these changes only take place in the result set. To insert this new row into the `course` table in the database requires one further step:

```
results.insertRow();
```

After the insert row has been inserted into the database, it is still not reachable within the result set by normal cursor movement unless the type parameter for the statement has been set to `TYPE_SCROLL_SENSITIVE`, which means that the result set has a scrollable cursor that is sensitive to changes to the database.

Note that what is referred to as the current row of the cursor is not affected by the cursor moving to the insert row – that is, the insert row does not become the current row – so having completed the insertion, the cursor can then be returned to where it was before being moved to the insert row by the following method:

```
results.moveToCurrentRow();
```

There are similar update methods that use the column number, as was the case for the get methods.

EXERCISE 3.10

Explain why the method `moveToCurrentRow` only has any effect when the cursor is on the insert row.



Deletion and update of existing rows in a database involve a simpler process, only requiring that the rows are first retrieved by some query to become part of a result set, then the result set can be processed by moving the cursor so that the current row can be deleted or updated as required. For example, we can delete the row we have just inserted, having chosen to give it a course code larger than all others, with the following set of statements:

```
results = st.executeQuery(
    "SELECT course_code, title, credit "
    + "FROM course "
    + "ORDER BY course_code ");
results.last();
results.deleteRow();
```

These statements assume that we have created a statement `st` with `TYPE_SCROLL_SENSITIVE`.



EXERCISE 3.11

Describe how these three Java statements achieve the required outcome of deleting the row that has just been inserted, explaining why the **ORDER BY** clause is necessary. Suggest an alternative way in which this outcome could have been achieved without this clause.

A more general way of deleting rows, rather than the specific context of Exercise 3.11, is to create a result set containing just those rows to be deleted and then processing each row in that result set using `deleteRow`.

Updating a table involves the same principle so, having generated a result set that includes the row to be updated and placed the cursor on it, one of the update methods such as those we showed for the insert row can be used to change the value in any column. Once the required changes have been made to the current row in the result set, the row in the database table can be updated by:

```
results.updateRow();
```



EXERCISE 3.12

When using Java to insert a row into a table, describe the general processing steps involved in using a prepared **INSERT** statement and in using a result set. Assume that the values to be inserted are provided by user input and held as variables.

To summarise updating with JDBC, the most important issue is to distinguish the two approaches:

- ▶ Execute an SQL update statement, either immediate or prepared, in which case the main difference from executing a query (as far as the Java program is concerned) is that the result is an integer.
- ▶ Execute a query to generate a result set, which is updated one row at a time and the change applied to the database.

Calling stored procedures

We have described how most kinds of SQL statement may be executed using JDBC, except for the stored procedures that are introduced in Section 7 of *Block 3*. Just now we are interested not in how one might be created, but in how one may be executed from a Java program.

SQL allows a stored procedure to be invoked by a call of the procedure with each input parameter assigned an appropriate argument. Such a call is regarded as an SQL statement, and some procedure calls can be immediately executed or prepared in the same way as other kinds of SQL statement. In particular, an argument in a procedure call can be a parameter marker and set in the same way as described for queries.

However, stored procedures have one feature that make them different from other kinds of SQL, which is the ability to return a value to a calling program as an output parameter. That feature has required a special form of a prepared statement to support procedure calls, provided by the class `CallableStatement` (a subclass of `PreparedStatement`).

EXERCISE 3.13

Explain why the get methods described for obtaining the results of queries cannot be used for obtaining the result from an **OUT** parameter of an SQL procedure call.



Thus, as shown to be required by the solution to Exercise 3.13, the class `CallableStatement` enables the value of an output parameter to be obtained by providing get methods for each of the data types, similar to the get methods for result sets. There is one difference in the way the get methods are used, and that is the SQL data type of each SQL **OUT** parameter has to be registered before executing the parameter call.

However, rather than examine the detail of how this is done, it is enough for you to recognise that JDBC enables all kinds of SQL statement to be executed from a Java program.

EXERCISE 3.14

Explain how JDBC interaction with a DBMS is not always expressed in terms of SQL statements.



3.2 SQLJ

This subsection gives an overview of **SQLJ**, which is embedded SQL for Java. In this case, there has been a reverse of the sequence of development we described in Section 2 for other languages, in which embedded SQL was followed by ODBC. For Java, first JDBC was developed by the Java community and then SQLJ was produced. This sequence of development has consequences that we shall examine later in this section, the most important of which is that SQLJ is defined in terms of JDBC.

SQLJ is specified as part of the SQL standard, where it is formally referenced as SQL/OLB – that is, SQL Object Language Bindings, though no other language apart from Java is considered. You should note that the term SQLJ originated as the name of the group that initially worked on Java and databases, including Java routines in a database that we shall consider in the next subsection, but now SQLJ is used more specifically to refer to embedded SQL for Java.

Since SQLJ is embedded SQL, in many respects it is directly comparable to the embedded SQL we described in Section 2. The general development and execution of SQLJ programs is broadly similar to that of embedded SQL. In particular, the stages shown in Figure 2.1 are required. The pre-compilation stage is known as SQLJ translation, during which SQLJ statements are converted into Java statements using JDBC.

Apart from a different way that SQLJ statements are distinguished from the host language, which requires an initial `#sql` with SQL embedded in curly brackets (braces), many statements appear exactly the same as we have described for embedded SQL, such as:

```
#sql {INSERT INTO course (course_code, title, credit)
      VALUES (:code, :title, :credit)};
```

This example has the same host variables as the equivalent example in Section 2, but host variables do not have to be specially declared for SQLJ; they only need to be declared as ordinary Java variables of an appropriate type to provide the values to be

inserted into the columns of the table. However, they still need the prefix `:` within an SQL statement to indicate that they are Java variables.



EXERCISE 3.15

Identify those kinds of SQL statement which can be treated in this way – that is, just giving an SQL statement with host variables – and those which cannot.

So, as with all other ways of using SQL within programs, it is queries returning many results that require a different treatment. In fact, SQLJ provides two ways of handling queries, both based on a kind of object called an **iterator** that provides the means to process a result set.

First, we consider the use of a **positioned iterator**. This requires an SQLJ declaration that a new iterator class is to process a result set with a specified set of data types for the columns. For example, to be able to process a result set that include all columns of the **course** table, we need the following declaration:

```
#sql iterator CoursePosIterator(String, String, int);
```

When translated, this SQLJ statement generates a class, which we have chosen to name as `CoursePosIterator`. We can then declare a variable of this iterator type, for example, `myPosIterator`, using a normal Java statement:

```
CoursePosIterator myPosIterator;
```

The result of a query with matching data types can be assigned to this iterator by the SQLJ statement:

```
#sqlj myPosIterator = {SELECT course_code, title, credit
                        FROM course};
```

This kind of iterator then allows you to fetch from each row of a result set into host variables in a similar way to embedded SQL, naming the iterator rather than a cursor:

```
#sqlj {FETCH :myPosIterator INTO :code, :title, :credit};
```

FETCH is an SQLJ statement that moves the cursor as in embedded SQL. This statement needs to be processed in a Java loop to access all rows, using a termination checking method `endFetch` on the iterator.



EXERCISE 3.16

Explain why `myPosIterator` does not have a prefix in the example statement with **SELECT** but has a prefix `:` in the example statement with **FETCH**.

The alternative to a positioned iterator is a **named iterator**, and the two iterators are similar in many respects. To begin with, a named iterator has to be declared but, in this case, the names of the columns have to be included, such as:

```
#sql iterator CourseNamIterator(String course_code,
                                String title, int credit);
```

Next, a variable of this generated class is declared:

```
CourseNamIterator myNamIterator;
```

Note that this is an SQLJ statement but does not contain an SQL statement and so does not need `{...}`.

Then the result of a query can be assigned to this variable:

```
#sqlj myNamIterator={SELECT course_code, title, credit  
FROM course};
```

Now comes the different bit. During translation of the declaration of a named iterator class, a method is generated for each column. Each method has a name identical to the column on which it is based, and it can be used to obtain the value for that column from the current row of the iterator, as in:

```
courseCode = myNamIterator.course_code();  
courseTitle = myNamIterator.title();  
credit = myNamIterator.credit();
```

To process all the rows of a result set, the method `next` is applied to the iterator to move the cursor and to test for the end of the result set, exactly the same as for JDBC.

EXERCISE 3.17

Compare and contrast the use of the terms 'position' and 'name' in the way JDBC and SQLJ process a result set.



As you can see from the solution to Exercise 3.17, JDBC and SQLJ use the terms 'position' and 'name' in rather different ways, and you need to be careful that you know the context in which they occur.

Having examined how SQLJ handles SQL statements, we return to consider the impact of the translation of SQLJ into JDBC. One consequence is that a source program may be written with a combination of SQLJ and JDBC statements. To support this mixed mode of working, there are methods to produce a JDBC result set from an SQLJ iterator and conversely to produce an iterator from a JDBC result set. Furthermore, since SQLJ requires a connection context, this can be obtained from a JDBC connection. A further consequence is that the resulting compiled SQLJ program is portable, which is different from embedded SQL (see Exercise 2.1).

We conclude this overview of SQLJ by considering some of the reasons for using it in preference to JDBC.

- ▶ SQLJ requires less source code than JDBC.
- ▶ SQLJ allows Java host variables (and, more generally, Java expressions) within statements, whereas JDBC requires separate getting and setting statements.
- ▶ SQLJ is static SQL, which allows statements to be checked during a translation, including the possibility of ensuring all table and column references are valid if a schema is accessible (and so is safer; see Exercise 3.5).

3.3 Other roles for Java

So far in this examination of the role of Java in accessing a database, we have adopted a simple model of a Java program interacting directly with a DBMS within a single machine environment. In this subsection, we briefly introduce some other ways in which Java may be used.

Database routines using Java

Section 7 of *Block 3* describes how SQL can be used to define routines – that is, functions and procedures – that are stored within a database and then executed by some SQL invocation. But that block is just concerned with SQL, and so gives you only half the story about database routines.

Instead of having a routine whose body is written using SQL, there is an alternative kind of routine in which the body is written using any of the programming languages for which embedded SQL defines the data type mapping. Such a routine is declared using SQL, so its specification is recorded by the DBMS and can be invoked by an SQL statement, but instead of having an SQL body it identifies the compiled code external to the database that is executed when the routine is invoked. For example, the essential elements of an SQL procedure `example_proc` implemented by an external routine `my_proc` written in the language C and kept in a library `my_library.dll` would be defined as:

```
CREATE PROCEDURE example_proc()
  LANGUAGE C
  EXTERNAL NAME 'my_proc@my_library.dll'
```

Note that this is an SQL statement that has to be executed in the same way as any other SQL, such as using an interactive tool like ISQL.

In this example, the external routine is specified as belonging to some library that is quite separate from the database, and while we have just given the file name for the library as being local to the database, it could be stored anywhere. Furthermore, the library is managed independently of the database, so there needs to be some form of control on how its routines might be changed. A procedure with an external body is invoked by an SQL **CALL** statement in the same way as a procedure with an SQL body, such as:

```
CALL example_proc()
```

You should appreciate that there are various other properties of routines that may need to be specified and, in particular, the data types of any parameters and, for functions, the data type of the returned value must be specified. However, further details are not relevant for our purpose, since the important point is to recognise that there is such a capability.

So what about Java? In principle, there is no reason why Java should not be used in the same way as other languages, but we have already seen that it has special treatment with JDBC and SQLJ, and it is also different for external routines. The most significant point is that DBMSs have been developed to incorporate a Java Virtual Machine, so that Java code can be installed in a database and executed by the DBMS. Thus an SQL routine using Java is not really ‘external’ at all.

The details of how to use Java in a database is specified by a part of the SQL standard called SQL Routines and Types using the Java programming language (SQL/JRT). One immediate concern of those accustomed to using Java might be that the language does not have routines – it has only methods for its classes, and the instance methods that you would normally write can only be invoked on an instance of the class. This does not fit very well in the way an SQL routine would be used, so a Java body for an SQL routine must be written as a class method (i.e. a static method, but using static in a different sense from static SQL).

Having written and compiled one or more Java classes with their associated static methods, the first step in enabling the methods to be used in a database is to create a **JAR file** – that is, a Java Archive file that packages a collection of classes in a special zipped format. The JAR file can then be installed in a database by using a stored procedure defined by the standard, **SQLJ.INSTALL_JAR** (where **SQLJ** is the name given to the schema in which the procedure must be stored). Supposing a JAR file

named `my_classes.jar` had been created and stored in `my_folder`, it could be installed in a database as shown in the following SQL statement:

```
CALL SQLJ.INSTALL_JAR(
  'file://localhost/c:/my_folder/my_classes.jar',
  'stored_jar', 0)
```

In this statement, the first argument is a string that specifies the location of the JAR file, the second argument is a name to be used for the JAR when it is stored in the database (which is different from the name of the file) and the third argument relates to the special 'deployment' requirements that we are not considering here.

Once the JAR has been installed in the database, functions and procedures can be defined in the same way as for other languages, except that the clause **EXTERNAL NAME** references an appropriate static method in a class within a JAR stored in the database. For example, if the database JAR `stored_jar` contains the class `a_class` having a static method `a_method`, that method can be specified as the body of a procedure as in:

```
CREATE PROCEDURE example_proc()
  LANGUAGE JAVA
  EXTERNAL NAME 'stored_jar:a_class.a_method'
```

Again, we must emphasise that there are various other properties that may need to be specified, particularly when a procedure or function has parameters – in this case, a clause **PARAMETER STYLE JAVA** is required as well as the specific data types of each parameter.

EXERCISE 3.18

The benefits of using SQL database routines described in Section 7 of *Block 3* also apply to Java database routines. Describe any specific advantages or disadvantages in using Java database routines as compared to SQL routines.



Object-relational mapping

In this subsection, we introduce a rather different approach to database access that has evolved in response to a particular problem for many Java programmers – that is, they may have to write Java programs requiring access to data in a database but do not want to learn SQL in order to do so.

There are various ways in which this problem might be tackled. One way is to organise a program into a number of layers so that what is often called a 'business logic' layer is separate from a 'storage' layer. Such a storage layer contains all the SQL (i.e. uses JDBC or SQLJ) required for access to a database, which is written by a database specialist, and then the business logic layer may be written using just Java with the storage layer providing whatever database data is needed via Java objects. However, the storage layer may provide nothing more than **CRUD methods** – that is, create, retrieve, update and delete – for each table, with no attempt to support more complex queries to match the requirements of the business logic.

Another way of tackling the problem, which does not involve writing any SQL, requires an **object-relational mapping** component. This component is a general-purpose software package that allows a Java application developer to view database data in terms of classes and objects, and to express the required use of that data using just Java. The purpose of such a mapping component is to translate between classes in a Java application process and tables in an SQL DBMS. Figure 3.1 shows how this new component relates to application processes and a DBMS.

In this situation, 'create' refers to adding a row, which in SQL terms is 'insert'.

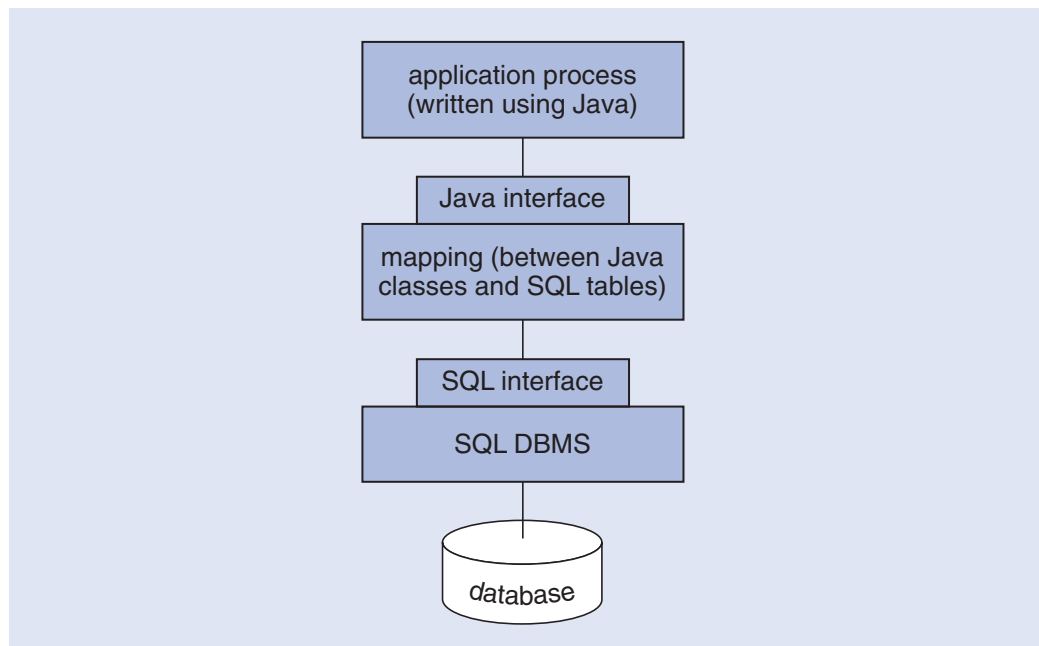


Figure 3.1 Model of database access with object-relational mapping

The DBMS in Figure 3.1 is no different from those to which you are now accustomed, and we have shown an SQL interface to encompass all kinds of DBMS interface that use SQL to express the database interaction. Such a DBMS is able to support other kinds of application process using JDBC or SQLJ. From the DBMS's point of view, the mapping component is effectively the same as any other user process in the way it uses SQL to interact with the DBMS.

The mapping component in Figure 3.1 should be considered as a kind of database tool, in that it is general-purpose software usable with any application data, and often with any DBMS, thus it is sometimes referred to as a **mapping tool**. There are many products available that can be considered as mapping tools, and there is significant variation amongst these products in how they provide the required capability of translating between classes and tables. Because of this variation, we are not going to examine any further detail of this approach but just introduce some general principles so that you can appreciate what is involved.

The most important aspect of object-relational mapping is the specification of the mapping – that is, how Java classes relate to tables in a database. Such a mapping needs to be specified not only for each database but for each application process, because the data requirements are likely to be different for each application. A mapping is used by a mapping tool in a way that is comparable to the use of a schema by a DBMS, and it is sometimes referred to as a **mapping schema**. Figure 3.1 does not show any mapping schema (nor database schema) but you should envisage one as being a file quite distinct from the database and used only by the mapping tool.

As an example of a very simple mapping, the table **course** can be mapped directly to a Java class that has instance variables corresponding to each of the columns of the table. However, as soon as you start including other tables and relationships between them, it begins to get a bit more complicated because associations between Java classes are represented by references to objects and not by values of foreign keys. Additionally, when you introduce Java subclasses, which a relational database does not support, you have more problems – or perhaps you can say that there are more choices to make in designing the mapping.

We are not going to examine what mappings are possible, but let us now consider how one might be specified. One significant factor is whether a given mapping tool

includes the means to create a mapping automatically, either by examining tables in a database and defining classes that represent the data in the tables for use within a Java program or, conversely, by examining the classes in a Java program and producing definitions of tables in which the state of objects can be stored.

If a mapping tool does not support such automatic creation of a mapping, then it has to be done manually by a person who understands both the way data is represented in a relational database and how it is represented within Java programs.

In whatever manner a mapping is produced, it has to be expressed in some form of syntax so that it can be kept in a file for use by the mapping tool when an application process needs to access the database. The mapping syntax differs for each kind of mapping tool, but it is often expressed using XML.

EXERCISE 3.19

Explain the possible disadvantage of a mapping tool that automatically produces table definitions based on examination of the classes in a Java program.



Servlets and Java Server Pages (JSP)

In this final role of Java in accessing databases, we broaden our view to a web environment in which people use a browser to send a request to a web server for some web page. The web page is returned to the browser for display to the user. Such a web page may be some unchanging document, such as a published report, or may be 'dynamic', in this case meaning that the data contained in the web page can vary according to the user and time. Usually, such dynamic data comes from a database, so we need to consider how a database can be accessed in this kind of environment.

One of the early uses of Java was in writing applets, which are bits of Java code invoked from a web page displayed in a browser. This Java code is executed in the browser environment (i.e. in the client machine), and so there are various restrictions on what an applet is allowed to do in order that a malevolent web server cannot provide one that corrupts the client machine. The main uses of applets are for producing special effects within the user interface, but they can be used for accessing a database. However, the restrictions on applets, the need for JDBC support within the client and communication from the applet to a remote database mean that applets are not regarded as a general solution for database access in a web environment.

So, instead of a client-side solution, a server-side solution was developed based on Java code called a **servlet**. A servlet is not an independent program, but has certain requirements to interface to a web server so that it is able to respond to requests for web pages. Apart from the need to construct the web page to be returned, a servlet can contain Java that provides data content for the web page by including JDBC or SQLJ statements. We are not considering any further details of servlets because the database access that they provide is expressed using the same Java capabilities we have described in previous sections.

An alternative kind of server-side solution is called **Java Server Pages**, or more commonly just JSP. As initially developed, a JSP page is essentially an HTML page containing Java **scriptlets** – that is, fragments of Java code, which can include JDBC or SQLJ statements, embedded in HTML. A JSP page is then translated into a servlet, which can be executed as we have just described.

JSP has further evolved; now there are certain extensions that do not even require any Java. Special XML tags have been defined that allow you to just write SQL statements within an HTML page, such as:

```
<sql:query var = "courses">
    SELECT * FROM course
</sql:query>
```

While this might appear to be much simpler than writing Java, executing the SQL (after it has been translated into JDBC) gives you a result set referenced by the variable `courses`, which still has to be processed one row at a time just as you would with JDBC. Thus the syntax of what you write is different, but the concepts required for understanding database access remain the same.

3.4 Summary

In summarising the way Java supports access to databases from application programs, it is important to recognise two significant aspects. One is the distinctive role of Java in being the only object-oriented programming language for which there are standards for database usage, specified as JDBC, SQLJ and SQL/JRT for database routines. This has ensured that many DBMSs support Java database access, as well as including a JVM within the DBMS. Java itself enables the portability of applications, so the overall impact has been that Java is the most widely used language for developing applications that access databases.

The second significant aspect is the variety of ways in which Java can be used in accessing a database. In this section we have focused on JDBC to give you details of what is involved in developing applications that retrieve and update a database. We have also aimed to help you recognise that there are other ways in which Java can be used – with SQLJ embedded in a Java program, with SQL routines that have a Java body, with an object-relational mapping tool that enables an application developer to write database applications without having to use SQL, and with Servlets and JSP in a web environment. Making choices as to which is the most appropriate way to develop applications is a complex task that is beyond the scope of this course, though we have considered some of the differences that need to be taken into account in making those choices.

4

Alternative approaches

In the previous sections, we have covered various approaches to database access that are applicable to different kinds of conventional programming languages, with particular emphasis on the special treatment of Java.

In this final section, we introduce a further perspective on database access that is based on using what are known generically as **scripting languages**. This term is used to refer to a programming language with some or all of the following characteristics:

- ▶ high-level: can be written quickly and concisely;
- ▶ dynamic: properties, such as data types, are determined during execution;
- ▶ interpreted: source code execution (i.e. no separate compilation);
- ▶ interactive: execution of individual statements.

There is a wide range of scripting languages, often developed for a specific and limited purpose, such as those used for system management (the origin of the term). Some scripting languages can be considered general-purpose, but their popularity is particularly associated with web-based applications where the speed of development and flexibility to change are important characteristics.

The scripting languages that have been widely used for database access include Perl, Python (and a version Jython for use with a Java Virtual Machine), PHP and Ruby (with a framework for databases called Rails). Each language has its vocal advocates, and each has its own way of expressing database access, except for the one common factor of using SQL. However, the basic principles are no different from those we have described. So, each language requires the following steps, some of which may be combined or may require several statements:

- ▶ connect to a database;
- ▶ formulate an SQL statement for the required database access;
- ▶ execute the statement;
- ▶ for queries returning multiple rows, use a cursor to process each row.

To demonstrate what is possible, we provide some examples of using Python, an object-oriented scripting language. For database access, it requires an additional module that implements the specification of the database application programming interface, DB-API. This specification has been produced, as has the rest of the Python language specification, by the Python Software Foundation. There are many implementations of the DB-API available and it is necessary to install one that is suitable for the DBMS to use, in the same way that ODBC and JDBC require a driver appropriate to a particular DBMS. There are also some that can work with any DBMS that supports ODBC, in a similar way to the JDBC–ODBC bridge that we described in Section 3.

The following examples are based on a DB-API module called mxODBC, which is freely available for non-commercial use. As you might infer from its name, mxODBC has been developed to work with an ODBC driver.

Python statements can be executed interactively, in which case the Python interpreter prompts on each line with `>>>`, or as a filed collection. For database access, it is necessary to start with an `import` statement to ensure that the specified database module is available for use, such as:

```
import mx.ODBC.Windows
```

There is an open source module called `pyodbc` that also works with an ODBC driver, but it does not support the DB-API specification in the way described in these examples.

In this and all subsequent statements, it is essential that letters have the right case, since Python is case sensitive.

The user identifier and password can also be included as further parameters, if necessary.

A DB-API module supports two main classes and associated methods. A connection object is created from the module as follows:

```
univ_conn = mx.ODBC.Windows.connect("UniversityDSN")
```

In this statement, `connect` is a method invoked for the imported module. It creates a connection to the database identified by the ODBC data source name `UniversityDSN`. This connection is an object that is referenced by the variable named `univ_conn`. You should note that while Python objects are typed, Python variables that reference them are not typed. So this variable could next be assigned any value we wished, such as a number.

Having established a connection object, it can be used to create what is referred to in Python's DB-API as a cursor by invoking the `cursor` method, as in the following statement:

```
univ_cursor = univ_conn.cursor()
```



EXERCISE 4.1

Explain to what kind of JDBC statement this Python statement is comparable.

A Python cursor object can then be used to execute SQL statements by invoking the `execute` method, as in the following:

```
univ_cursor.execute("INSERT INTO course \nVALUES ('c9', 'Testing', 30)")
```

The symbol `\` indicates a continuation line.

EXERCISE 4.2

Explain how this use of a Python cursor is different from the way the term is used in JDBC.

As can be seen from the solutions to Exercises 4.1 and 4.2, Python uses some of the JDBC terms with a different meaning, so you need to be careful in your interpretation.

Let us now consider the situation where a JDBC cursor would be used in processing a result set after executing a query, which for Python would be expressed as in the following example:

```
sql = "SELECT course_code, title, credit FROM course"
univ_cursor.execute(sql)
```

On completion of this `execute` method for the given SQL statement, the cursor object `univ_cursor` holds the result of the query – that is, a result set in JDBC. A cursor object supports several methods to access the results it holds, the simplest being `fetchone`, which does what it says – it fetches one row from the cursor object. However, an important feature of Python is that it has a built-in data structure for a tuple, and we can assign a row to a variable as a single value that is implicitly typed as tuple, as shown in the following example:

```
a_row = univ_cursor.fetchone()
```

Having obtained a row, it is necessary to decide what to do with it. To display it interactively, Python allows you to simply type the variable `a_row` and its value is displayed. For this example, you would see: `('c2', 'Syntax', 30)`.

However, if you wanted to display specific columns or process those columns in some way, you would have to extract each column value from the tuple data structure. The

only way to do that is by using an index number to reference a position in the tuple. Index numbers start at 0 and are specified as `[...]`, so you can type `a_row[0]` and find `'c2'` is displayed.

EXERCISE 4.3

In what way does the requirement to reference columns by position violate relational principles?



It is also necessary to consider how to display or process each of the rows in the result, and for this reason you need to remember that fetching a row in ODBC or JDBC implicitly moves the cursor to the next row. In such cases, the cursor acts as a pointer to a row in a result set. The same principle applies for Python, so you can repeat the above assignment of a row using the method `fetchone` to obtain each of the rows in the result. Python provides various means of iteration by which this can be done.

Another possibility is to obtain all rows in one go, using the method `fetchall`. Actually, it gives you all the remaining rows – that is, all those that have not yet been fetched. Again, Python allows you to assign the result to a single variable, such as:

```
all_rows = univ_cursor.fetchall()
```

The value assigned to the variable `all_rows` is structured as a list of tuples, where a list is another built-in data type similar to a tuple. The variable `all_rows` can be displayed with all the column values in all the rows presented as a sequence across the screen, so that it does not appear as a table. A specific row in the list can be referenced in the same way as a particular value in a tuple by using an index number, such as `all_rows[3]`. Remember that such a row is a tuple and may be handled in the same way as we described for the variable `a_row`.

However, having obtained all the rows of a result set, you can then use the high-level features of Python to process it. For example, you can write the following statement to display all the rows separately, where the variable `a_row` takes the value of each row in `all_rows` in turn:

```
for a_row in all_rows:
    print a_row
```

There are many more details about the use of Python for database access, such as how you can specify parameters for an SQL statement, but we are not developing these examples any further. We hope the simple examples provided demonstrate two main issues about scripting languages.

The first issue relates to the nature of scripting languages and how they might be used. Being high-level means that a single statement is capable of doing something that would require many statements in a low-level language such as Java. You have seen an example of this in the built-in data types that allow a complete row or collection of rows to be assigned as a single value. Being dynamic means that a variable does not have a predefined data type – it simply references an object that can be any type. Being interpreted means that execution can be immediate, which also supports interactive working that enables you to see the effect of individual statements. This way of working is very productive in a developmental situation, where you want immediate feedback on what you are doing. Having tested a collection of statements, they can be filed and executed as a complete program, but execution is likely to be slower than for a compiled program in another language. However, it is also possible to consider this as a way of testing SQL statements, which can then be implemented by transferring to, say, a Java program.

The second issue is the comparison with JDBC, particularly in terms of a retrospective reflection on the low-level use of Java. It is hoped that you were aware of the continual need to describe that each Java variable must be defined as a particular type, even though explicit syntax is not given. However, a more significant issue is that separate JDBC statements are required to obtain a value for each column in a row, and it is necessary to know each column's data type in order to specify the appropriate method for this purpose. While such a low-level approach does provide more detailed control of what a program is doing, it is time-consuming to write programs, and testing can be done only by the compilation and execution of complete programs rather than individual statements.

Topic summary

In this topic on application development you have been presented with a lot of detail so in this summary we aim to identify the more important aspects for you to remember, which are mainly focused on the general processing principles rather than any specific language features. While Java has been examined in detail, so that you are able to start using JDBC in practice if you wish, many of the main concepts are directly comparable to other programming languages.

Thus SQL statements can be embedded in any programming language, with the consequential need for a pre-compilation stage in order to convert the source code into a form acceptable to a compiler. This may be contrasted with using an application programming interface (API), either ODBC or JDBC, which can be directly compiled and results in a portable application process. However, an API allows dynamic SQL, which can have dangers in comparison to static embedded SQL.

An important requirement for any application process is to be able to execute an SQL statement repeatedly with varying input. Embedded SQL supports this requirement using host variables, characterised by a : prefix, whereas an API allows the construction of a dynamic SQL statement or parametrisation of a prepared statement.

For retrieval of many rows there is a common requirement to use a cursor mechanism to fetch each row in turn. In addition, embedded SQL again uses host variables in a single fetch statement, with SQLJ also enabling methods to be used, whereas an API requires each column value to be obtained separately from a fetched row.

This retrieval process introduces another general aspect of developing application processes, namely the contrasting view of a database when compared to the use of interactive SQL. *Block 3* describes a logical processing model for the execution of an SQL query, which is expressed entirely in terms of a succession of intermediate tables to give a final result that is a table. The view of data for processing the result of an SQL query is at a much lower level, working row by row and, for an API, handling each column value separately.

Apart from embedded SQL and use of an API, there are other approaches to application development that may be considered in some situations, such as using servlets or JSP for a web environment or scripting languages for high-level interactive development. While there is a need to be careful about the terminology and capabilities of these alternative approaches, they do not introduce any significantly different processing principles for database access.

LEARNING OUTCOMES

After studying this topic, you should be able to:

- ▶ Describe the general context and concepts of application development for databases.
- ▶ Describe the concepts and explain the programming tasks in using embedded SQL for application development.
- ▶ Describe the concepts and explain the programming tasks in using JDBC for application development.
- ▶ Distinguish the use of embedded SQL from the use of an application programming interface such as JDBC in application development.
- ▶ Identify alternative ways in which Java can be used to access databases.
- ▶ Describe the possible benefits of scripting languages for accessing databases.

Solutions to Topic 2 Exercises

SOLUTION 2.1

Only the source code for an embedded SQL program is portable because the pre-compilation processing is different for each DBMS and results in object code that will only work with that DBMS – that is, it is not portable.

Transferring an embedded SQL program to another vendor's DBMS requires the original source code to be processed using that vendor's pre-compiler, and then compiled to produce another version of the object code.

SOLUTION 2.2

- (a) The first form of the **INSERT** statement gives explicit values for the columns of a single row, which would involve a program in transferring a sequence of values for that row, and so it is valid as an embedded statement.
- (b) The second form of the **INSERT** statement specifies that many rows are selected by a query, which the DBMS can do without requiring any data from an application program, and so this form of the statement is also valid as an embedded statement.

SOLUTION 2.3

The name `var` must be declared as a host variable in an SQL **DECLARE SECTION** within the program, with a data type that is compatible with its use in the SQL statement.

SOLUTION 2.4

```
EXEC SQL
UPDATE course
SET credit = :points
WHERE title = :title;
```

SOLUTION 2.5

The stopping condition for this loop needs to use the special variable `SQLSTATE`, checking when its value indicates that there is no data (02000).

SOLUTION 2.6

An ODBC driver converts database requests expressed in terms of the ODBC interface into requests expressed in terms of the native interface for the DBMS being used. The ODBC driver can do this only if the request is 'appropriate' – that is, if it is written so that it can interact with the native interface of that DBMS.

SOLUTION 2.7

Embedded SQL uses host variables to transfer data from a DBMS to a program. They can be placed in an **INTO** clause of a query, and data is transferred according to the corresponding position of the host variables and the columns in the statements, with no association by name but appearance within the same statement showing the mapping between names.

ODBC uses either the `SQLGetData` routine after `SQLFetch` or a binding before `SQLFetch` to specify a column number and a variable into which the data is transferred. A column is named in the SQL statement that is executed, but all subsequent references to it are by means of column number, so there is no obvious association between columns and variables.

SOLUTION 3.1

The `executeQuery` method returns a result set with its associated cursor automatically opened.

SOLUTION 3.2

A result set is the result of executing a query, which retrieves rows in some order. An **ORDER BY** clause can be included in the query, otherwise the order is implementation-dependent.

SOLUTION 3.3

The DBMS can apply the condition if you change the text of the SQL query in the `executeQuery` method to include an appropriate **WHERE** clause for the column `credit` (i.e. add: `WHERE credit < 40`), otherwise the processing of the result set remains the same.

The Java program can apply the condition if you change the processing of the result set to include a test on the variable `credit` (i.e. add: `if (credit < 40)`) so that the course details are listed only for those rows that satisfy this condition.

SOLUTION 3.4

The value of the variable `sql` is the string

```
SELECT course_code, title, credit FROM course WHERE credit < 10
```

SOLUTION 3.5

An embedded SQL program is written with explicit SQL statements referencing known tables, and specified columns within those tables, compiled so that those statements cannot be changed. This fixed behaviour can be subject to testing and approval.

A dynamic SQL program can be written in such a way that its access to a database is determined when the program is executing by constructing queries that may involve any data (subject to access controls within the database). It is possible that the behaviour of such a program may not be fully predetermined and so cannot be tested properly for approval.

SOLUTION 3.6

```
sql = "SELECT title, credit "  
      + "FROM course "  
      + "WHERE course_code = ?";  
prepSt = univConnection.prepareStatement(sql);  
prepSt.setString(1, code);  
results = prepSt.executeQuery();  
results.next();  
courseTitle = results.getString("title");  
credit = results.getInt("credit");  
System.out.println(courseTitle + " "  
                    + String.valueOf(credit));
```

SOLUTION 3.7

The variable `result` should have a value of 1, since one row should have been added to the course table.

SOLUTION 3.8

```
sql = "UPDATE course "  
      + "SET title = ? "  
      + "WHERE course_code = ? ";  
prepSt = univConnection.prepareStatement(sql);  
prepSt.setString(1, newTitle);  
prepSt.setString(2, code);  
result = prepSt.executeUpdate();
```

SOLUTION 3.9

The restrictions are essentially the same, because the reason for the restrictions is the same – there has to be a direct mapping back to a base table so that changes to the database are well defined. That is, a query for this purpose cannot include joins, **GROUP BY**, **DISTINCT** or functions.

SOLUTION 3.10

For any row other than the insert row, the location of the cursor determines the current row and hence there can be no change.

SOLUTION 3.11

The `results` result set contains all the rows from the course table, ordered by course code so that the new row is known to be the last row. The cursor can be moved to the last row and then the row is deleted, achieving the required outcome. Without the **ORDER BY** clause, you do not know where the row to be deleted might be, so the result set would have to be processed row-by-row, checking the value of the course code to see whether it was c9 and deleting the row when it matched.

SOLUTION 3.12

When inserting a row, both approaches involve the execution of a single SQL statement, but in one case it is an **INSERT** statement and in the other it is a query. A prepared statement for **INSERT** needs parameter markers that have to be set before execution using the variables holding data input.

The alternative approach requires that the query generates a result set from the table to be updated, for which the insert row is then updated using the variables holding the input data. In addition, the insert row has to be inserted into the database.

SOLUTION 3.13

The get methods described for queries are applied to a result set, whereas a parameter is a property of the statement and so requires methods applicable to statements.

SOLUTION 3.14

Updating with a result set requires a final step in which each change to a row of the result set is applied to the database. This involves one of the methods `insertRow`, `updateRow` or `deleteRow`, and they all request a DBMS to make a change to the database without it being expressed as an SQL statement.

SOLUTION 3.15

INSERT, **UPDATE** and **DELETE** statements, and queries that return a single row, can all be treated in this way; queries that return many rows cannot.

SOLUTION 3.16

`myPosIterator` is declared as a Java variable, and in the statement with **SELECT** it is an ordinary variable being assigned the result of executing the SQL query given in `{ ... }`, but in the statement with **FETCH** it is used as part of the SQL (i.e. it is within `{ ... }`) and in that context requires : to show it is a host variable.

SOLUTION 3.17

For JDBC, 'position' and 'name' are alternative ways of referencing the columns in a result set, so methods such as `getString` have two equivalent versions, one taking an integer argument that is a column number and the other taking a string argument that is a column name.

For SQLJ, 'position' and 'name' relate to the two kinds of iterator that can be used to process a result set. A positioned iterator supports **FETCH**, which enables column values to be assigned to host variables according to their corresponding positions in a list. A named iterator supports methods having the name of columns in a result set, which enables column values to be obtained from the iterator.

SOLUTION 3.18

The main advantage is that Java is a much more powerful language than SQL for general handling of numeric or textual data (Java has many built-in libraries of methods), and so is more appropriate for complex processing.

Another advantage is that programmers who are familiar with Java can write database routines without having to learn the SQL control statements needed for more complex requirements.

A significant disadvantage is that a database administrator cannot examine a Java routine in a database to find out what it does and assess how it may affect the operation of a database (particularly necessary in case of errors). There is a rather indirect route back to the Java source code, and even then a DBA might not necessarily understand the details of the Java code.

SOLUTION 3.19

The result would be tables for storing data from the Java program in a database, but based on the requirements of just that one program without any consideration of the requirements of other programs that might want to share that data. Designing a database should take all application requirements into account to enable data sharing.

SOLUTION 4.1

For a given connection, JDBC provides a method to create a `Statement` object.

SOLUTION 4.2

A JDBC cursor is the means by which a result set is processed one row at a time, whereas the **INSERT** statement being executed in this Python example has no result set.

SOLUTION 4.3

The properties of relations given in *Block 2* include the rule that the position of a column is not significant and, consequently, you should refer to a column only by its name.

Index

A	F	ODBC data source	60
absolute path	FLWOR expression	ODBC driver	60
23	forest	ODBC manager	60
application process	31	OUTPUT command	42
50	function		
application programming interface (API)	78	P	
59	H	parameter marker	69
attribute	host language	parent node	13
16	host variable	positioned iterator	76
C	54	pre-compiler	55
character data	HTML	prepared statement	62, 68
13	9	procedure	78
child node	I	processing instruction	21
13	insert row	programming language	50–51, 53
comment	72	publishing function	32
21	iterator		
compiler	76	R	
54	J	Relax NG	19
constructor function	JAR file	result set	61, 65, 69
32	78	root node	13
context node	Java	routine	78
23	50–51, 64	row number	67
context path	Java Server Pages	rowset	61
24	81	S	
CRUD method	Java Virtual Machine	Schematron	20
79	64, 78	scripting language	83
current row	JDBC	scriptlet	81
66, 73	51, 64	serialization	15
cursor	JDBC driver	servlet	81
58, 61, 66, 84	64	SGML	9
D	JDBC–ODBC bridge	shredding	35
database tool	65	source code	52
50	JSP	SQL Anywhere	29, 42
Document Object Model (DOM)	81	SQL routine	58
23	L	SQL/CLI	59
document prolog	leaf node	SQL/JRT	78
20	13		
document type declaration	M		
20	mapping		
Document Type Definition (DTD)	30		
18	schema		
dynamic SQL	80		
62	tool		
E	80		
edge table	markup language		
36	8		
element	markup tag		
13	9		
embedded SQL	N		
53, 75	named iterator		
empty element	76		
13, 30	native interface		
encoding	54		
20	node		
end-tag	13		
13	O		
entity	object code		
19	52		
export	object-relational mapping		
30, 42	79		

- SQL/XML 29
- SQLJ 64, 75
- start-tag 13
- static method 78
- static SQL 62, 78
- stored procedure 74
- stylesheet 21, 24
- T
- traversal 15
- tree 13
- U
- Unicode 20
- V
- valid XML 18
- W
- well-formed XML 14
- World Wide Web Consortium (W3C) 8
- X
- XHTML 25
- XML 8
 - application 12, 21
 - data type 39
 - declaration 20
 - parser 21
 - schema 19
 - value 39
 - well-formed 14
- XMLAGG 34
- XMLCONCAT 40
- XMLELEMENT 32
- XMLFOREST 34
- XMLPARSE 39
- XMLQUERY 41
- XMLSERIALIZE 42
- XMLTABLE 38, 40
- XPath 23
- XQuery 24, 38, 41
- XSL-FO 24
- XSLT 24

